



Технически университет – Варна

Факултет по изчислителна техника и автоматизация

Катедра: Софтуерни и интернет технологии

Дисциплина: Обектно-ориентирано програмиране – 1 част

Курсов проект на тема

XML Parser

На *Фатменур Бейтиева Ердинчева*

факултетен номер: 24621627

специалност: Софтуерни и интернет технологии

курс: II

група: 1 а

2026

Задание на курсовия проект

Да се напише програма, реализираща четене и операции с XML файлове.

Характеристиките на XML елементите, поддържани от програмата, да се ограничат до:

- идентификатор на елемента
- списък от атрибути и стойности
- списък от вложени елементи или текст

Да се поддържат уникални идентификатори на всички елементи по следния начин:

- Ако елементът има поле "id" във входния файл и стойността му е уникална за всички елементи от файла, да се ползва тази стойност.
- Ако елементът има поле "id" във входния файл, но стойността му не е уникална за всички елементи от файла, да се ползва тази стойност, но към нея да се конкатенира някакъв низ, който да допълни идентификатора до уникален низ. (например, ако два елемента имат поле id="1", то единият да получи id="1_1", а другият - id="1_2")
- Ако елементът няма поле "id" във входния файл, да му се присъедини уникален идентификатор, генериран от програмата.

След като приложението отвори даден файл, то трябва да може да извършва посочените по-долу операции, в допълнение на общите операции (open, close, save, save as, help и exit):

print	Извежда на екрана прочетената информация от XML файла (в рамките на посочените по-горе ограничения за поддържаната информация). Печатането да е XML коректно и да е "красиво", т.е. да е форматирано визуално по подходящ начин (например, подчинените елементи да са по-навътре)
select <id> <key>	Извежда стойност на атрибут по даден идентификатор на елемента и ключ на атрибута
set <id> <key> <value>	Присвояване на стойност на атрибут
children <id>	Списък с атрибути на вложените елементи
child <id> <n>	Достъп до n-тия наследник на елемент
text <id>	Достъп до текста на елемент
delete <id> <key>	Изтриване на атрибут на елемент по ключ
newchild <id>	Добавяне на НОВ наследник на елемент. Новият елемент няма никакви атрибути, освен идентификатор
xpath <id> <XPath>	операции за изпълнение на прости <u>XPath 2.0</u> заявки към даден елемент, която връща списък от XML елементи

Минимални изисквания за поддържаните XPath заявки

1. Да поддържат оператора / (например "person/address" дава списък с всички адреси във файла)
2. Да поддържат оператора [] (например "person/address[0]" дава адреса на първия елемент във файла)
3. Да поддържат оператора @ (например "person(@id)" дава списък с id на всички елементи във файла)
4. Оператори за сравнение = (например "person(address="USA")/name" дава списък с имената на всички елементи, чиито адреси са "USA")

Забележка: За проекта не е позволено използването на готови библиотеки за работа с XML. Целта на проекта е да се упражни работата със структурирани текстови файлове, а не толкова със самия XML. Внимание: Не се изисква осигуряване на всички условия в XML и XPath спецификациите! Достатъчно е файловете да "приличат на XML" (както файла в горния пример, който не е валиден XML), а заявките да "приличат" на XPath.

- да се реализират XML namespaces
- да се реализират различните XPath оси (ancestor, child, parent, descendant...)

Съдържание

Задание на курсовия проект	2
Съдържание	4
Глава 1. Увод	5
1.1 Описание и идея на проекта	5
1.2 Цел и задачи на разработката.....	5
Глава 2. Преглед на предметната област	6
2.1. Основни концепции и алгоритми	6
2.2. Подходи и методи за решаване на поставените задачи	7
2.2.1. Прилагане на Обектно-ориентирания подход (ООП)	7
2.2.2. Организация на входно-изходните операции и управление на файлове (File I/O)	8
2.2.3. Модели и Шаблони за дизайн (Design Patterns)	8
2.3 Функционални и нефункционални изисквания	10
Функционални изисквания (Потребителски изисквания):.....	10
Нефункционални изисквания:.....	10
Глава 3. Проектиране	11
3.1. Обща структура на проекта (пакети)	11
3.2. Диаграми / Блок схеми на поведението	13
Глава 4. Реализация.....	24
4.1 Реализация на класове	24
Група 1: Информационен модел на данните (Пакет models)	24
Група 2: Команден интерфейс и операции (Пакет commands)	27
Група 3: Синтактичен анализ и валидация (Пакет parsers и exceptions).....	30
Група 4: Енджин за XPath заявки (Пакет xpath).....	32
Група 5: Входно-изходни операции и сериализация (Пакет io).....	34
4.1.1. Алгоритми и оптимизации	35
Глава 5. Тестване.....	40
5.1. Планиране и методология на тестването	40
5.2. Описание на тестови сценарии и примери за вход и изход	41
5.2.1. Тестове за управление на файловата система и автоматична корекция.....	41
5.2.2. Тестове за базова навигация и модификация	43
5.2.3. Тестове за стандартни XPath заявки с филтри и индекси	44
5.2.4. Тестове за сложни XPath оси (Axes) и Namespaces	45
Глава 6. Заключение.....	46
6.1. Обобщение на изпълнението на началните цели.....	46
6.2. Архитектурни изводи и принос на шаблоните за дизайн	47
6.3. Насоки за бъдещо разширение на софтуерната система.....	48
Използвана литература	49

Глава 1. Увод

1.1 Описание и идея на проекта

С бързото развитие на информационните технологии, необходимостта от стандартизирани формати за съхранение и обмен на данни става все по-голяма. XML (eXtensible Markup Language) е един от най-широко разпространените стандарти за структуриране на информация, използван в уеб услуги, конфигурационни файлове и бази данни.

Идеята на настоящия курсов проект е да се проектира и реализира конзолно базирано приложение (парсър) на езика Java, което позволява зареждане, четене, навигация, модификация и запазване на XML документи. Приложението е изградено изцяло чрез принципите на обектно-ориентираното програмиране (ООП). Основната идея е да се създаде напълно автономен XML парсър, написан изцяло на Java, без използване на външни библиотеки (като DOM, SAX или JDOM), което демонстрира дълбоко разбиране на структурите от данни и алгоритмите за работа с дървовидни йерархии. Приложението поддържа йерархична структура на данните, уникални идентификатори на елементите, търсене чрез XPath изрази и работа с XML Namespaces (именни пространства).

1.2 Цел и задачи на разработката

Основната цел на проекта е създаването на стабилна, разширяема и оптимизирана система за управление на XML файлове чрез интерфейс с команден ред (CLI).

За постигането на тази цел са дефинирани следните **основни задачи**:

- Проектиране на вътрешно представяне на XML документа чрез дървовидна структура от обекти.
- Гарантиране на уникалност на атрибутите `id` за всеки елемент спрямо зададените в заданието правила.
- Реализация на модул за четене (десериализация) и запис (сериализация) на XML текстови файлове.
- Разработване на система от команди за базова навигация и модификация (отваряне, затваряне, добавяне на възли, изтриване на атрибути, промяна на текстово съдържание).
- Реализация на специализиран подмодул за изпълнение на заявки, базирани на XPath стандарта, позволяващ сложно търсене по елементи, индекси и атрибути.
- Разширяване на функционалността с поддръжка на XML пространства от имена (Namespaces) и XPath оси (Axes - *child*, *parent*, *ancestor*, *descendant*).

Целта на разработката е да се приложат на практика принципите на Обектно-ориентираното програмиране (ООП) — капсулация, наследяване, полиморфизъм и абстракция.

Глава 2. Преглед на предметната област

2.1. Основни концепции и алгоритми

XML (eXtensible Markup Language) представлява де факто стандарт за маркиране, съхранение и пренос на данни, чиято вътрешна структура стриктно наподобява йерархично дърво. Основната концепция зад този стандарт е, че всеки XML елемент се разглежда като възел (Node), който може да притежава един родител, списък от деца (поделемента), набор от атрибути (двойки ключ-стойност) и текстово съдържание. За успешната реализация на парсър се използва итеративен алгоритъм, базиран на структурата от данни Стек (Stack), който позволява линейно време за обхождане на файла и лесно проследяване на текущия отворен таг. При изпълнението на заявки за търсене (XPath) се прилагат специализирани алгоритми за търсене в дълбочина и навигация по дървото нагоре и надолу чрез така наречените оси (Axes).

За успешната реализация на софтуерния проект е необходимо дефинирането на следните ключови концепции от компютърните науки и предметната област на XML:

- **Дървовидна структура (*Tree Data Structure*):** Тъй като XML е изцяло йерархичен формат, документът се представя в оперативната памет като дърво. Всеки XML таг е възел (Node/Element), който съдържа списък от своите деца (наследници) и референция към своя родител. За намиране на специфични елементи, както и за рекурсивната сериализация на документа обратно във файл, се използват алгоритми за обхождане на дърво в дълбочина (*Depth-First Search - DFS*).

- **Синтактичен анализ (*Parsing*):** Процесът на преобразуване на плоския текстов низ в обекти в паметта. Алгоритъмът за четене на файла трябва да разпознава отварящи и затварящи тагове, атрибути и текстово съдържание, използвайки регулярни изрази (*Regex*) и вградени методи за ефективна манипулация на символни низове (*String manipulation*).

- **XPath (*XML Path Language*):** Стандартизиран език за заявки, използван за навигация в XML документи. Алгоритмите за неговата реализация включват разделяне на заявката на отделни стъпки (*steps*) и последователно филтриране на списъци от възли въз основа на зададени критерии (индекси, атрибути или тагове).

- **Пространства от имена (*XML Namespaces*):** Концепция, създадена за избягване на конфликти при наличие на еднакви имена на тагове в един и същ документ, чрез използване на уникални префикси (напр. `bk:book`). Програмата имплементира алгоритми за резолвиране на локалното име, което позволява интелигентно филтриране дори при наличието на сложни декларации на пространства от имена.

2.2. Подходи и методи за решаване на поставените задачи

Проектирането и реализацията на софтуерната система изискват прилагането на софтуерни подходи, които гарантират гъвкавост, четимост на кода, лесна поддръжка и възможност за бъдещо разширение без фундаментална промяна в архитектурата. Тъй като основното ограничение на проекта изключва използването на готови външни библиотеки за обработка на XML, решението на задачата се базира изцяло на чистия обектно-ориентиран подход, съвременните стандарти за работа с файлови потоци и утвърдени шаблони за дизайн (Design Patterns).

2.2.1. Прилагане на Обектно-ориентирания подход (ООП)

Архитектурата на софтуера е изградена около четирите фундаментални принципа на обектно-ориентираното програмиране:

- **Капсулация (Encapsulation):** Защитата на състоянието на обектите и скриването на вътрешната им имплементация е критична стъпка при работа с йерархични структури от данни. В класовете `XmlElement` и `XmlDocument` всички ключови полета (като таг, текстово съдържание, референция към родител, списък с деца и глобалният регистър) са декларирани като частни (*private*). Достъпът до тях е строго контролиран чрез публични методи (*getters* и *setters*). За да се предотврати нежелана външна модификация на структурата на дървото, методите `getChildren()` и `getAttributes()` в `XmlElement` не връщат директни референции към вътрешните колекции, а капсулират данните чрез недекструктивни, немодифицируеми обвивки (`Collections.unmodifiableList` и `Collections.unmodifiableMap`). По този начин се гарантира, че структурата може да се променя само чрез специално дефинираните за целта методи като `addChild()`.

- **Абстракция (Abstraction):** Сложността на различните подсистеми е скрита зад чисти интерфейси и абстрактни класове, които дефинират договори (*contracts*) за поведение. Интерфейсът `ElementParser` абстрахира процеса на синтактичен анализ, позволявайки на главната програма да работи с парсъри, без да се интересува от техния конкретен алгоритъм. Абстрактният клас `Command` изолира логиката на интерфейса с команден ред от конкретното изпълнение на всяка потребителска операция, а абстрактният `XpathOperator` дефинира общата рамка за навигация по осите, скривайки специфичните филтриращи механизми.

- **Наследяване (Inheritance):** Принципът на наследяване е използван за изграждане на йерархия от преизползваем код. Всички конкретни команди (като `OpenCommand`, `ChildCommand`, `XPathCommand`) наследяват базовия клас `Command`. Това позволява споделянето на общи характеристики, като управление на аргументите, съхранение на синтактичното описание и името на командата, като по този начин се избягва дублирането на код (*DRY - Don't Repeat Yourself*). По сходен начин `IndexOperator` и `AttributeFilterOperator` наследяват `XPathOperator`, преизползвайки базовия метод `resolveAxisAndTag`.

- **Полиморфизъм (Polymorphism):** Използван е полиморфизъм по време на изпълнение (*Run-time polymorphism / Dynamic binding*). В главния цикъл на `CommandLineInterface`, системата управлява колекция от тип `Map<String, Command>`. Програмата не знае каква конкретна команда е въвел потребителят (дали е за изтриване, отваряне или търсене), но извиква полиморфния метод `command.execute()`, което активира правилната имплементация в съответния подклас. Същият полиморфен подход е приложен в `XPathService`, където се обхожда списък от оператори и се извиква техният метод `apply()`, осигурявайки динамично филтриране на XML елементите.

2.2.2. Организация на входно-изходните операции и управление на файлове (File I/O)

Работата с файловата система е организирана чрез съвременния стандарт **Java NIO (New I/O)**, предоставящ оптимизирани механизми за работа с потоци и файлови пътища. За постигане на архитектурна чистота е приложено разделение на отговорностите:

- **Изолиране на I/O логиката:** Класът `FileHandler` капсулира изцяло ниско ниво операции с диска, използвайки `Files.readString()` за бързо и ефективно зареждане на целия текстов съдържател в паметта и `FileWriter` за неговия запис. Моделът `XmlDocument` не комуникира директно с операционната система, а делегира тези задачи на `FileHandler`.

- **Работа в оперативната памет (In-memory computation):** Системата прилага модел на работа, сходен с транзакционната сигурност. При извикване на командата `open`, XML файлът се прочита веднъж, десериализира се от `XmlParser` и се изгражда пълно дърво от обекти в RAM паметта. Всички последващи модификации (добавяне на тагове чрез `newchild`, промяна на стойности чрез `set` или изтриване чрез `delete`) се извършват единствено вътре в оперативната памет върху обектите от тип `XmlElement`. Оригиналният файл на диска остава непокътнат и защитен от повреда в случай на неочаквано спиране на програмата. Промените се персистират (записват на твърдия диск) единствено при изрично извикване на командите `save` или `saveas`, които задействат класа `XmlSerializer` за рекурсивно генериране на XML структурата.

2.2.3. Модели и Шаблони за дизайн (Design Patterns)

За решаването на специфичните архитектурни предизвикателства в проекта са интегрирани четири класически шаблона за дизайн: (Таблица 1)

- **Command Pattern (Шаблон "Команда"):** Капсулира потребителската заявка като самостоятелен обект, съдържащ цялата необходима информация за нейното изпълнение. Позволява на `CommandLineInterface` да изолира обекта, който задейства командата, от обекта, който знае как да я изпълни. Шаблонът улеснява добавянето на нови функционалности към конзолния интерфейс, отговаряйки на *Open/Closed* принципа от *SOLID* принципите.

• **Registry Pattern (Шаблон "Регистър"):** Тъй като XML документите изискват често търсене на елементи по техния уникален id атрибут, линейното обхождане $O(n)$ на цялото дърво при всяка команда би довело до сериозно забавяне. Чрез класа IdAssigner в XmlDocument се изгражда централизиран регистър (Map<String, XmlElement>). Всеки елемент се регистрира със своето уникално ID още при зареждането, което оптимизира времето за търсене до константна сложност $O(1)$.

• **Strategy Pattern (Шаблон "Стратегия"):** Приложен е на две независими нива в проекта:

1. *При синтактичния анализ:* Главният XmlParser сменя динамично стратегията си за парсане в движение. Ако обработваният таг съдържа интервали, се активира AttributeTagParser за извличане на атрибути по схемата key="value". Ако няма интервали, се активира SimpleTagParser за чисти тагове.

2. *При XPath навигацията:* Различните видове синтактични стъпки (филтриране по стойност, вземане по индекс, стандартна навигация) са дефинирани като отделни стратегии (IndexOperator, AttributeFilterOperator, TagNavigationOperator), капсулиращи съответните алгоритми.

• **Chain of Responsibility (Шаблон "Верига от отговорности"):** Реализиран е в ядрото на XPathService. Услугата поддържа подреден списък (верига) от наличните XPath оператори. Когато се анализира конкретна стъпка от заявката, тя се прекарва през веригата. Всеки оператор проверява синтаксиса чрез метода canHandle(). Първият оператор, който разпозна синтактичния модел (например IndexOperator, ако има квадратни скоби []), поема пълната отговорност за обработката на тази стъпка и прекратява по-нататъшното обхождане на веригата.

Шаблон	Основни класове	Предназначение
Command Pattern	Command, OpenCommand, XPathCommand, SetCommand	Капсулиране на потребителските команди като обекти
Strategy Pattern	XPathOperator, IndexOperator, AttributeFilterOperator	Динамичен избор на алгоритъм за обработка
Chain of Responsibility	XPathService	Последователна обработка на XPath оператори
Registry Pattern	XmlDocument, IdAssigner	Бързо търсене на елементи по ID
Rich Domain Model	XmlElement	Капсулиране на бизнес логика в домейн обектите

Таблица 1: Използвани шаблони за дизайн

2.3 Функционални и нефункционални изисквания

Функционални изисквания (Потребителски изисквания):

- Системата трябва да поддържа интерактивен конзолен режим, приемащ следните команди:
 - Управление на файлове: `open <file>`, `close`, `save`, `saveas <file>`, `help`, `exit`.
 - Принтиране и навигация: `print`, `children <id>`, `child <id> <n>`, `text <id>`.
 - Извличане на данни: `select <id> <key>`, `xpath <id> <XPath>`.
 - Модификация: `set <id> <key> <value>`, `delete <id> <key>`, `newchild <id>`.
 - Системата трябва да поддържа навигация чрез XPath оси (`parent::`, `ancestor::`, `descendant::`, `child::`).
- Автоматично генериране на уникални `id` атрибути при отваряне на файл (чрез добавяне на суфикси като `_1`, `_2` или генериране на `gen-1`).
- Поддръжка на XML Namespaces

Нефункционални изисквания:

- Надеждност: Приложението трябва да прихваща изключения (Exceptions) при невалиден вход или липсващи файлове, без да прекъсва работата си неочаквано.
- Чистота на кода: Задължително спазване на конвенциите за писане на Java код, поставяне на JavaDoc коментари за всички класове и методи.
- Ограничения: Забранено е използването на външни библиотеки за парсане.
 - Приложението е конзолно.
 - Написано на Java
 - Приложението трябва да работи за Java 17 или по-нова версия
 - Лесно за разширяване и поддръжка.

Глава 3. Проектиране

3.1. Обща структура на проекта (пакети)



Архитектурата на приложението е разделена на логически слоеве, организирани в пакети, което гарантира ниска свързаност (*low coupling*) и висока кохезия (*high cohesion*) на компонентите. Всички класове в последователност от пакети `bg.tu-varna.sit.f24621627`, разделени в подпакети:

- Пакет **models** (модели на данните): Съдържа основните структури от данни – `XmlDocument` (представящ целия файл и неговия контекст) и `XmlElement` (градивният блок на дървото, съдържащ таг, атрибути, деца и референция към родител).

- Пакет **io** (вход/изход): Отговаря изцяло за комуникацията с файловата система. Класът `FileHandler` изолира операциите по четене и запис на низове, а `XmlSerializer` рекурсивно преобразува дървото от обекти обратно в добре форматиран XML текст.

- Пакет **parsers** (парсъри): Съдържа логиката по синтактичен анализ. Използван е абстрактен интерфейс `ElementParser`, реализиран от специализирани парсъри (`SimpleTagParser`, `AttributeTagParser`) и главния `XmlParser`, който управлява стека при изграждане на йерархията. Тук се намира и `IdAssigner`, отговарящ за генерирането на уникални идентификатори.

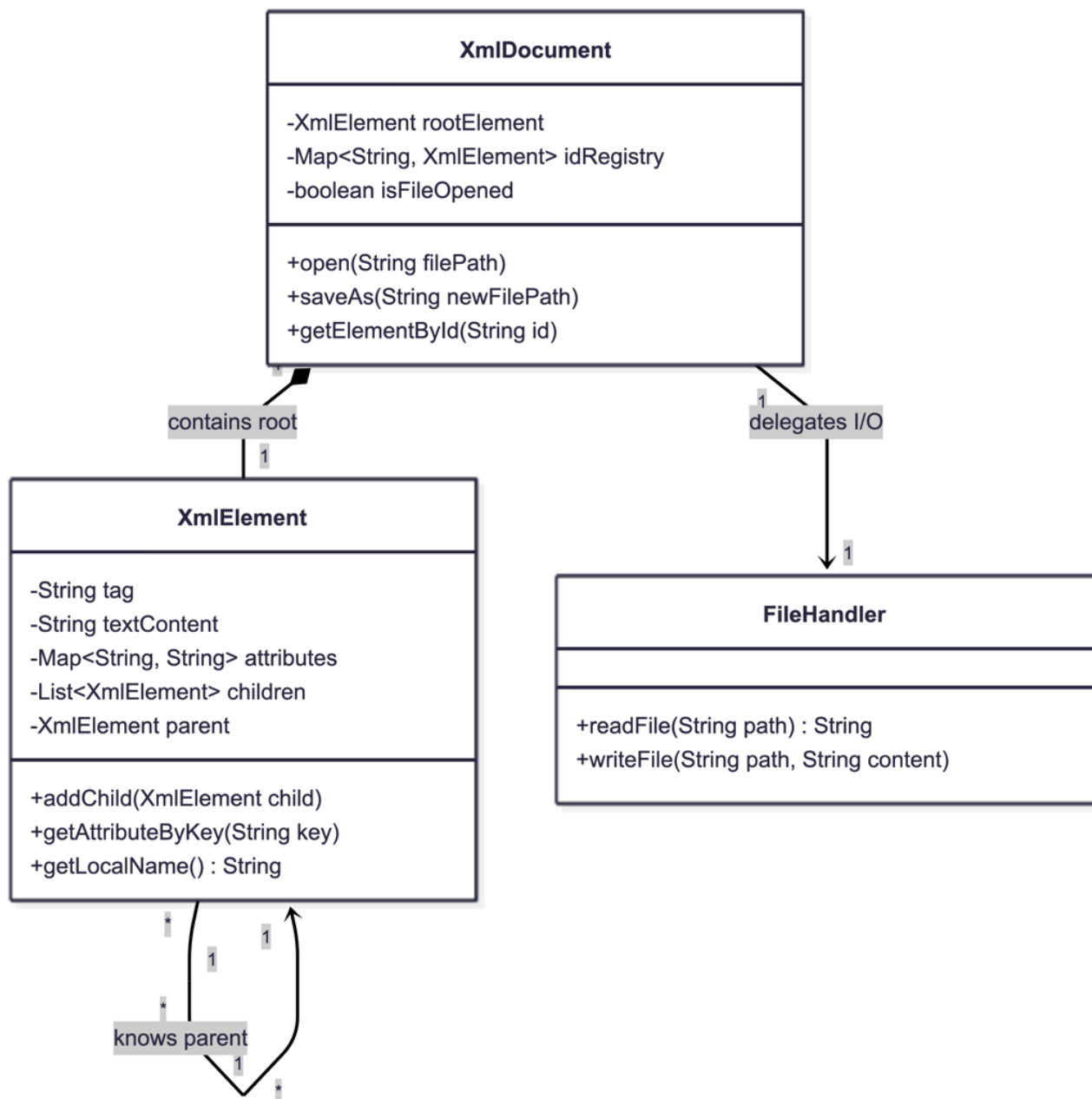
- Пакет **commands** (команди): Реализира шаблона *Command*. Всеки клас (напр. `OpenCommand`, `ChildCommand`, `XPathCommand`) наследява абстрактния базов клас `Command` и имплементира метода `execute()`.

- Пакет **xpath** (XPath логика): Капсулира сложната логика за търсене. Базиран е на шаблона *Strategy*, където `XpathService` координира различни оператори (`IndexOperator`, `AttributeFilterOperator`), а `AxisResolver` осигурява навигация по осите (родител, наследник и др.).

- Пакет **exceptions** (изключения): Съдържа персонализирани изключения като `XmlParseException`, за да се разграничат системните грешки от грешките във формата на файла.

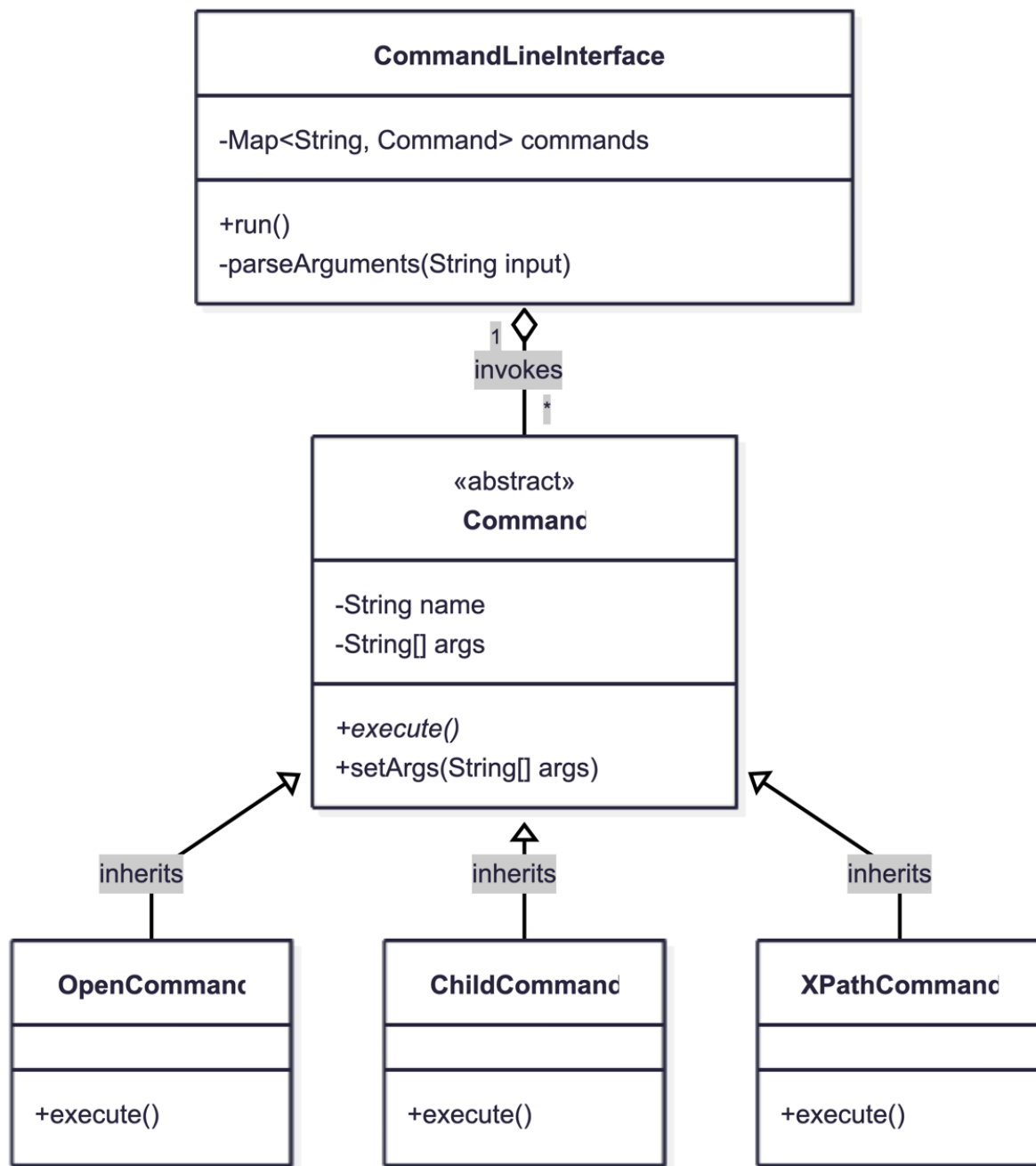
Входната точка (`Application`) няма собствен пакет, за да е лесно достъпна, заедно с `CommandLineInterface`, където е логиката за приемане на команди.

3.2. Диаграми / Блок схеми на поведението



Фигура 1. UML Клас диаграма на информационния модел.

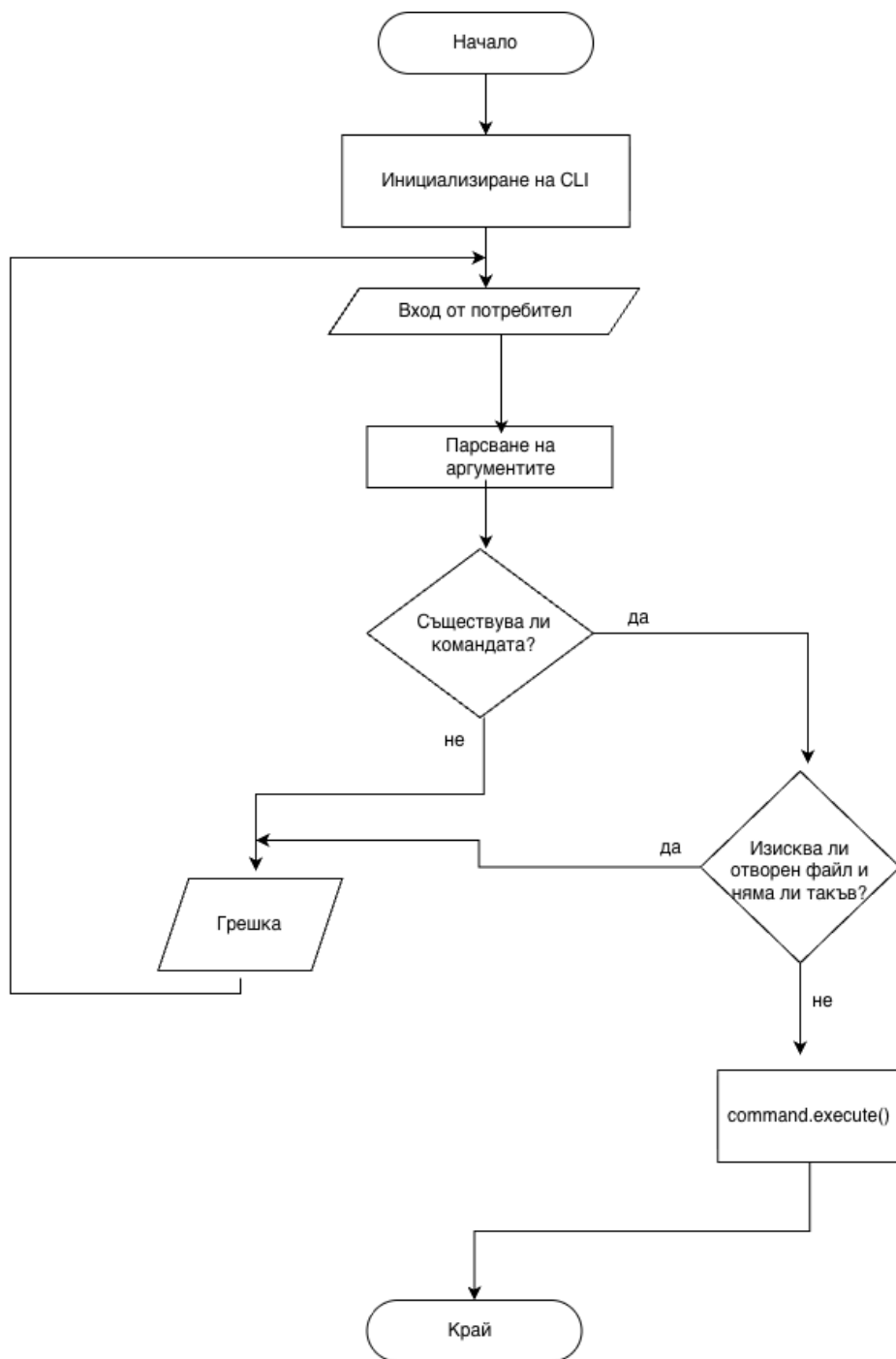
На представената диаграма е визуализирана структурата на основните класове, отговарящи за съхранението на XML документа в оперативната памет. Класът **XmlDocument** играе ролята на главен контейнер, който съхранява референция към корена на дървото (**root**) и поддържа глобалния регистър за бързо търсене по идентификатор (**idRegistry**). Класът **XmlElement** имплементира рекурсивна структура от данни (дърво), като всеки възел съдържа списък от своите деца и референция към своя родител. Операциите по четене и запис са изнесени в класа **FileHandler**.



Фигура 2. UML Клас диаграма, илюстрираща шаблона *Command*.

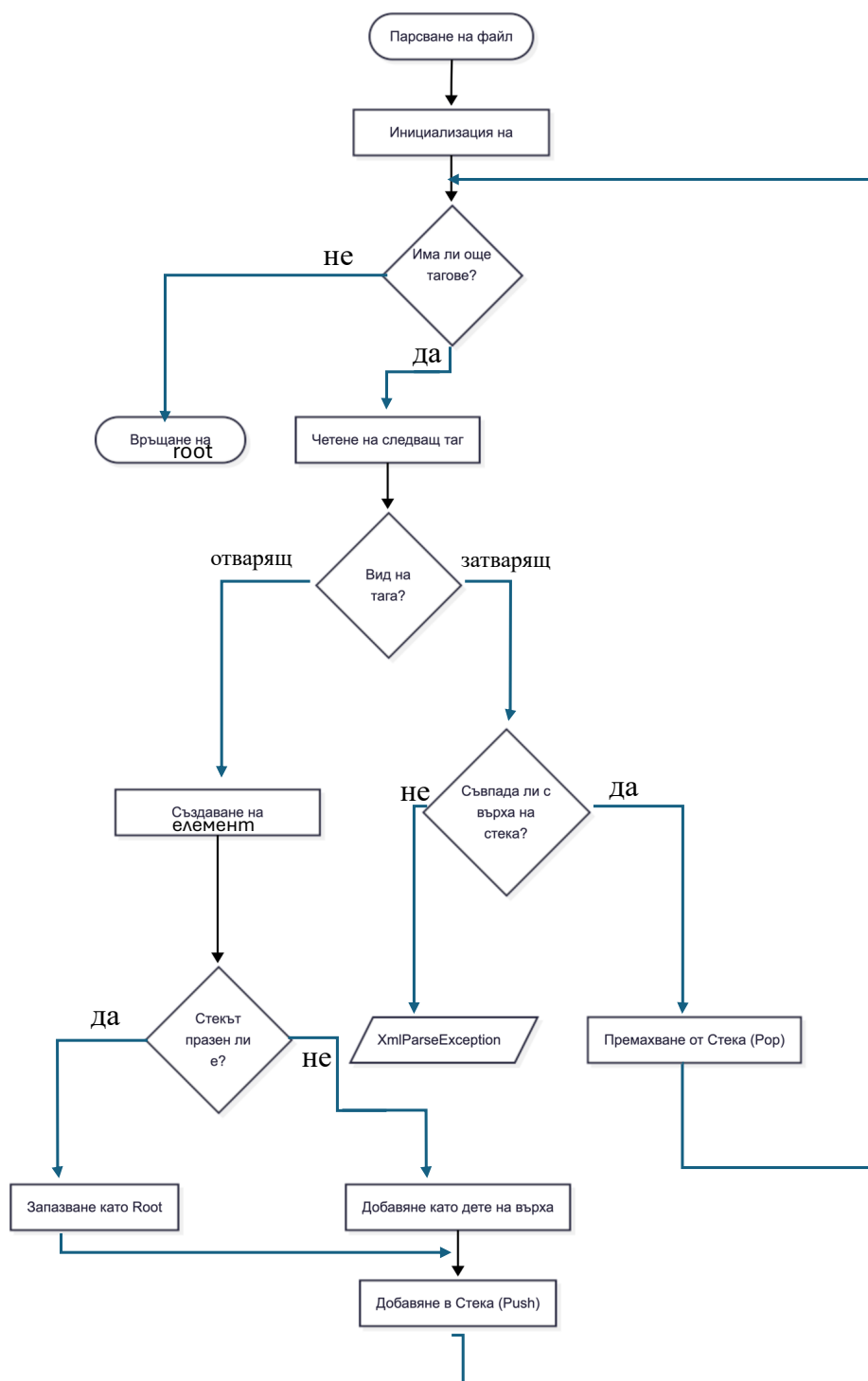
Тази диаграма показва архитектурното решение за управление на потребителския вход чрез шаблона за дизайн *Command*. Класът *CommandLineInterface* действа като инициатор (*Invoker*) и съхранява всички налични команди в хеш-таблица (*Map*), което позволява тяхното лесно извикване по име, без да се използват дълги вериги от *if-else* конструкции.

Всички конкретни операции (като *OpenCommand*, *ChildCommand* и *XPathCommand*) наследяват абстрактния базов клас *Command*. Те задължително реализират метода `execute()`, който капсулира специфичната логика за валидация и изпълнение. Този полиморфен подход гарантира, че добавянето на нови команди в бъдеще няма да изисква промяна в основния цикъл на програмата.



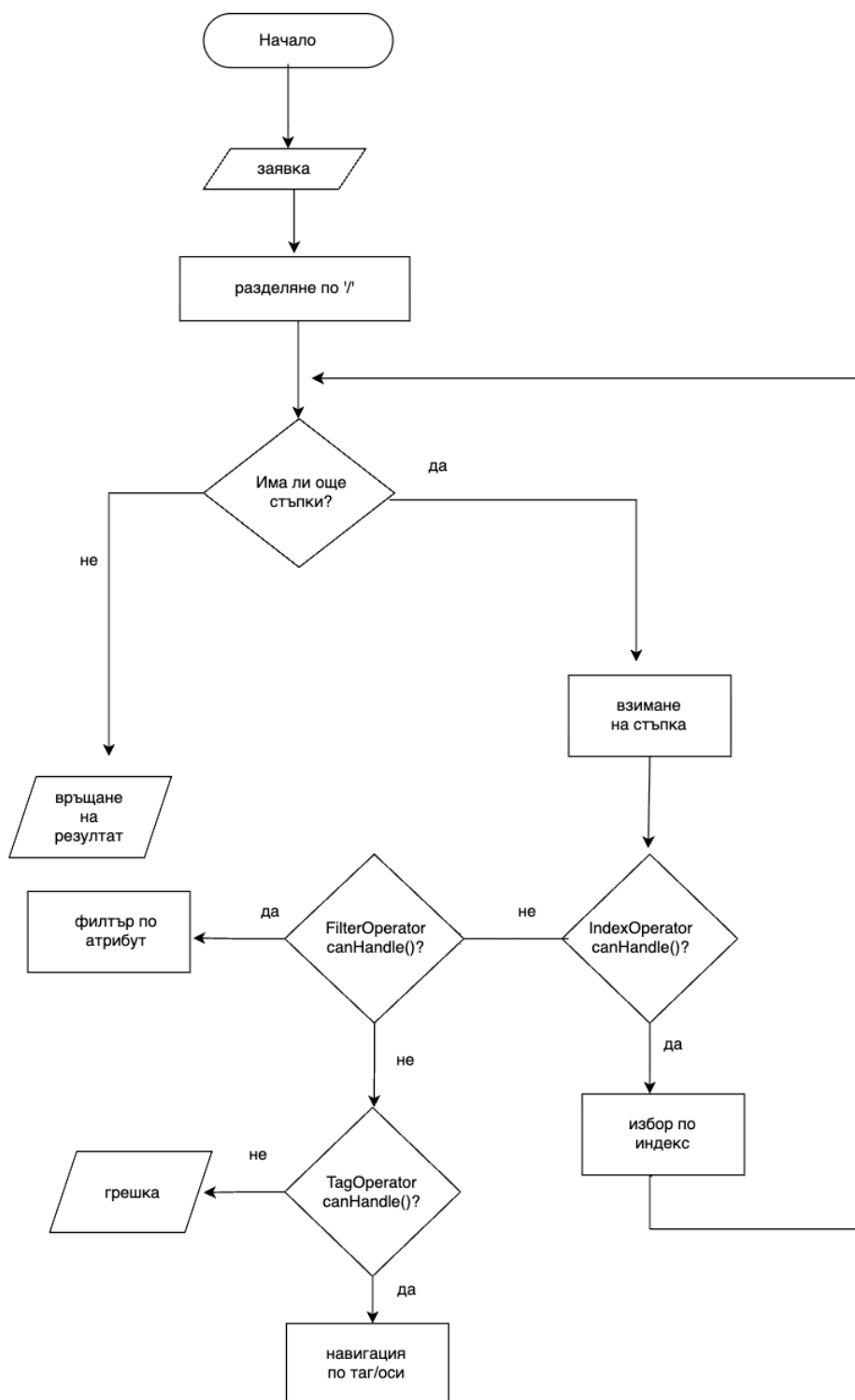
Фигура 3. Блок-схема на основния жизнен цикъл на програмата.

Схемата проследява алгоритъма на безкрайния цикъл за обработка на потребителски команди. При стартиране на програмата се инициализира командният интерфейс и се преминава в режим на изчакване на вход. След въвеждане на текст, системата разделя низа на аргументи и проверява дали въведената команда присъства в регистъра. Преди същинското изпълнение се извършва валидация на контекста – проверява се дали командата изисква предварително зареден XML файл (например командите за търсене и модификация). След успешно изпълнение чрез полиморфния метод `execute()`, резултатът се извежда на екрана или се отразява върху структурите в паметта, след което цикълът се затваря в очакване на следваща инструкция.



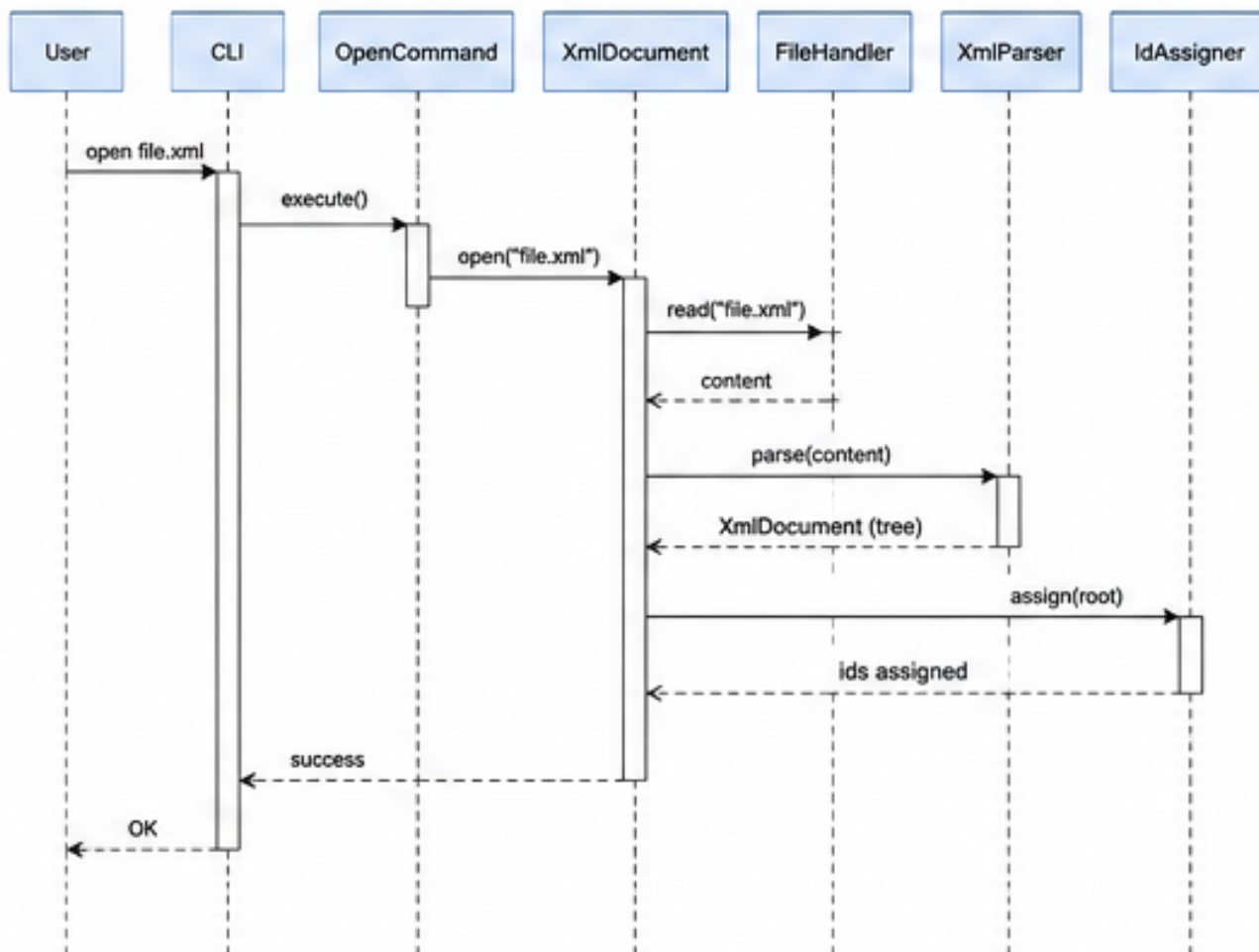
Фигура 4. Блок-схема на алгоритъма за синтактичен анализ (Parsing).

За изграждането на дървовидната йерархия от плоския XML текст е реализиран итеративен алгоритъм, базиран на структурата от данни Стек (Stack). Процесът обхожда текстовия низ линейно. При откриване на отварящ таг, програмата инстанцира нов обект от тип `XmlElement`, свързва го като дете на текущия елемент на върха на стека и го добавя (Push) в стека. Когато алгоритъмът срещне затварящ таг, той извлича върха на стека (Pop) и валидира дали имената им съвпадат. Този подход гарантира строга проверка за балансираност на таговете и предпазва системата от препълване на програмния стек (StackOverflow) при обработка на изключително дълбоко вложени XML документи.

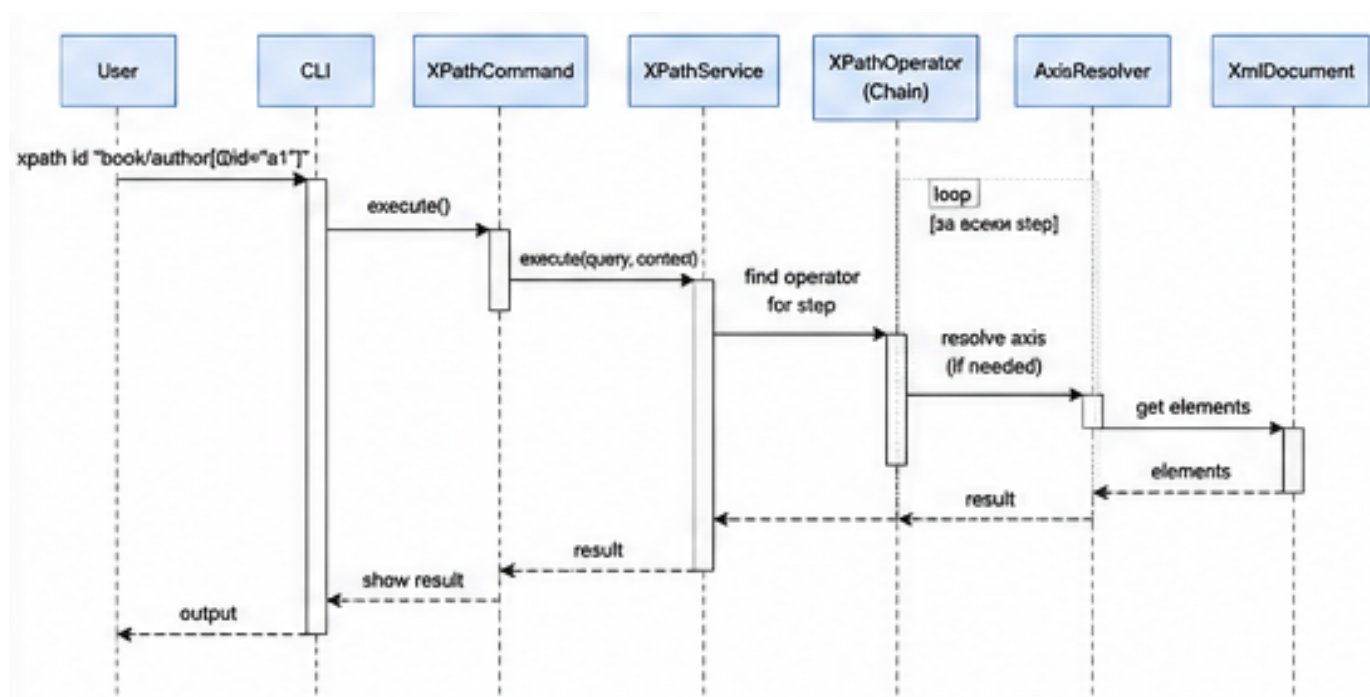


Фигура 5. Блок-схема на изпълнението на XPath заявка (Chain of Responsibility).

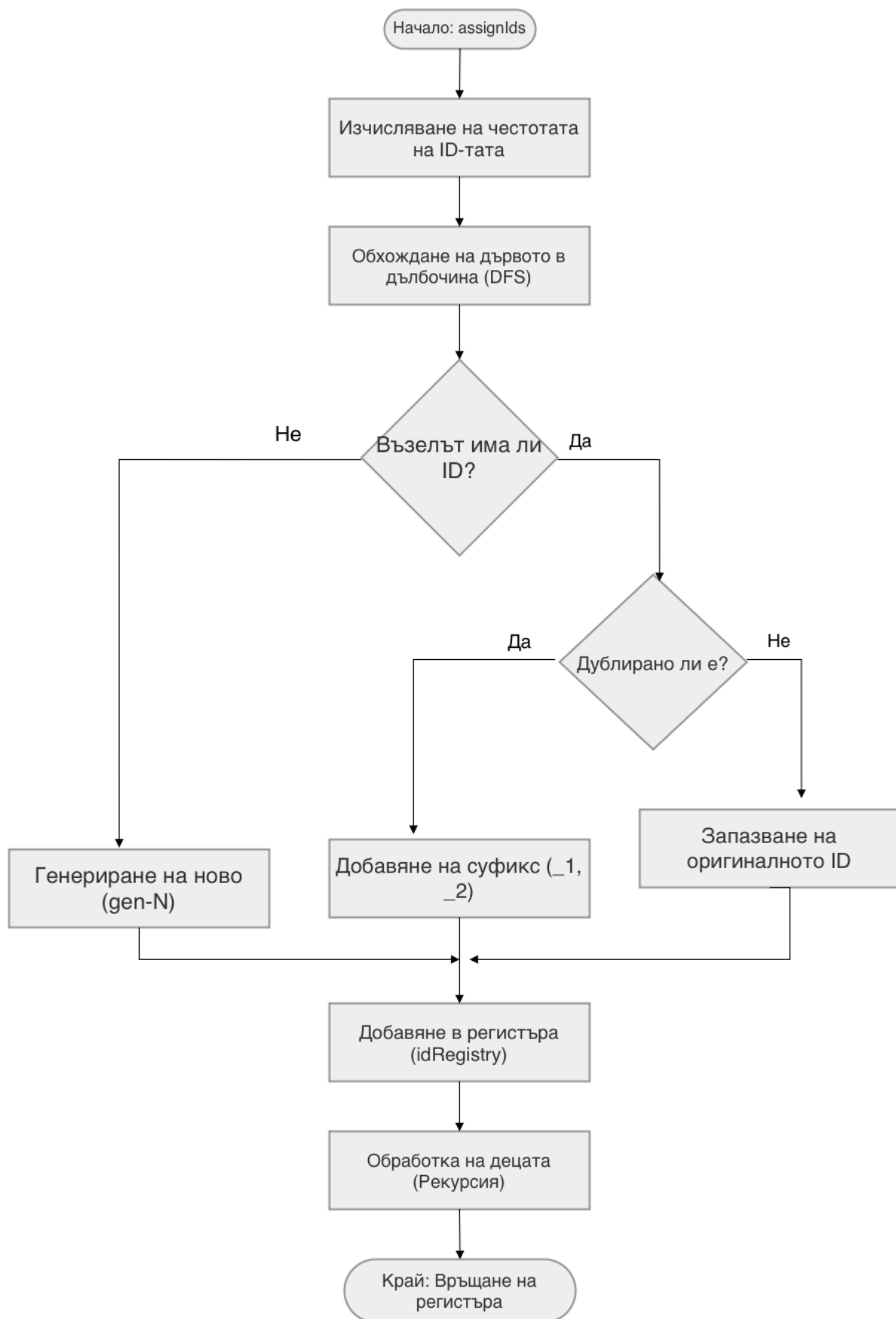
Сложната логика по обработка на XPath изрази е разбита на последователни фази. Заявката първо се разделя на отделни стъпки чрез разделителя /. За всяка стъпка алгоритъмът прилага шаблона "Верига от отговорности". Системата итерира през списък от предварително дефинирани оператори (стратегии), питайки всеки един дали разпознава синтаксиса чрез метода `canHandle()`. Ако стъпката съдържа квадратни скоби, контролът се предава на `IndexOperator`. Ако съдържа кръгли скоби, се извиква `AttributeFilterOperator`. Всяка стъпка подава своя резултат (списък от възли) като вход за следващата стъпка, което позволява изграждането на изключително сложни и верижни филтриращи механизми.



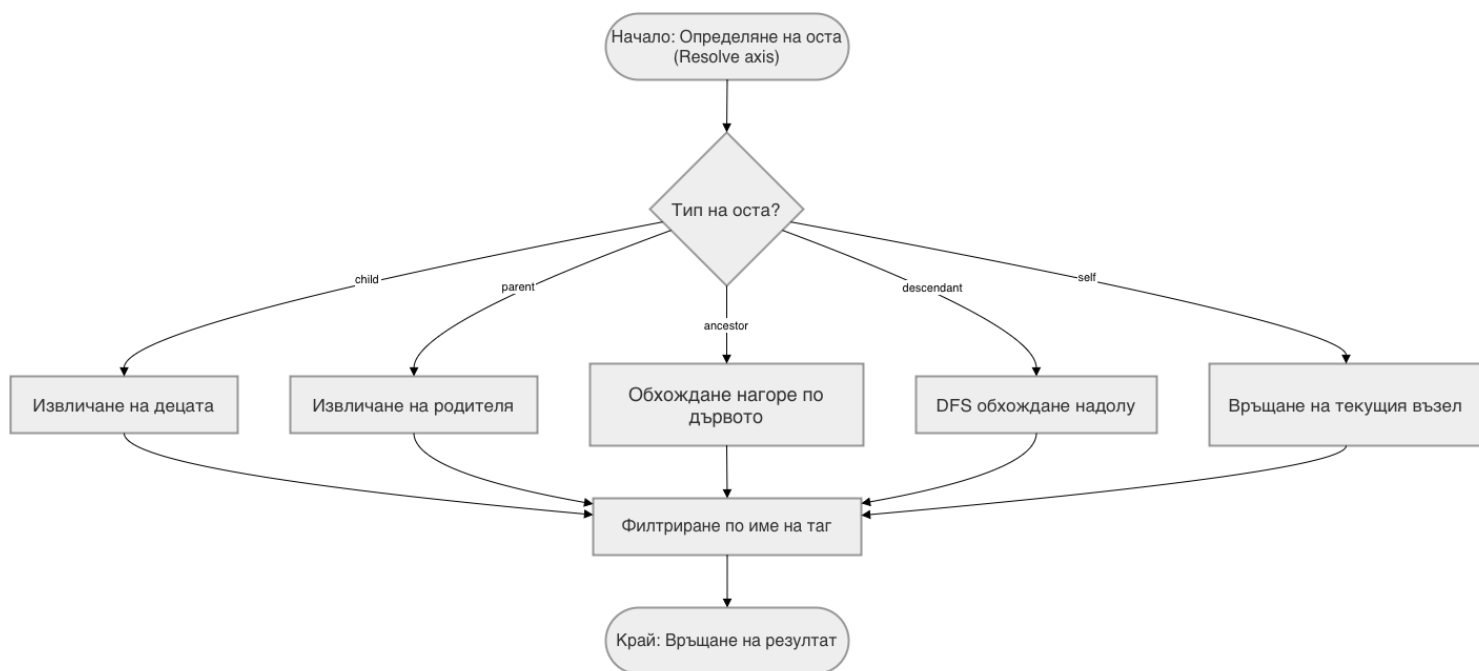
Диаграма 1: Отваряне на файл



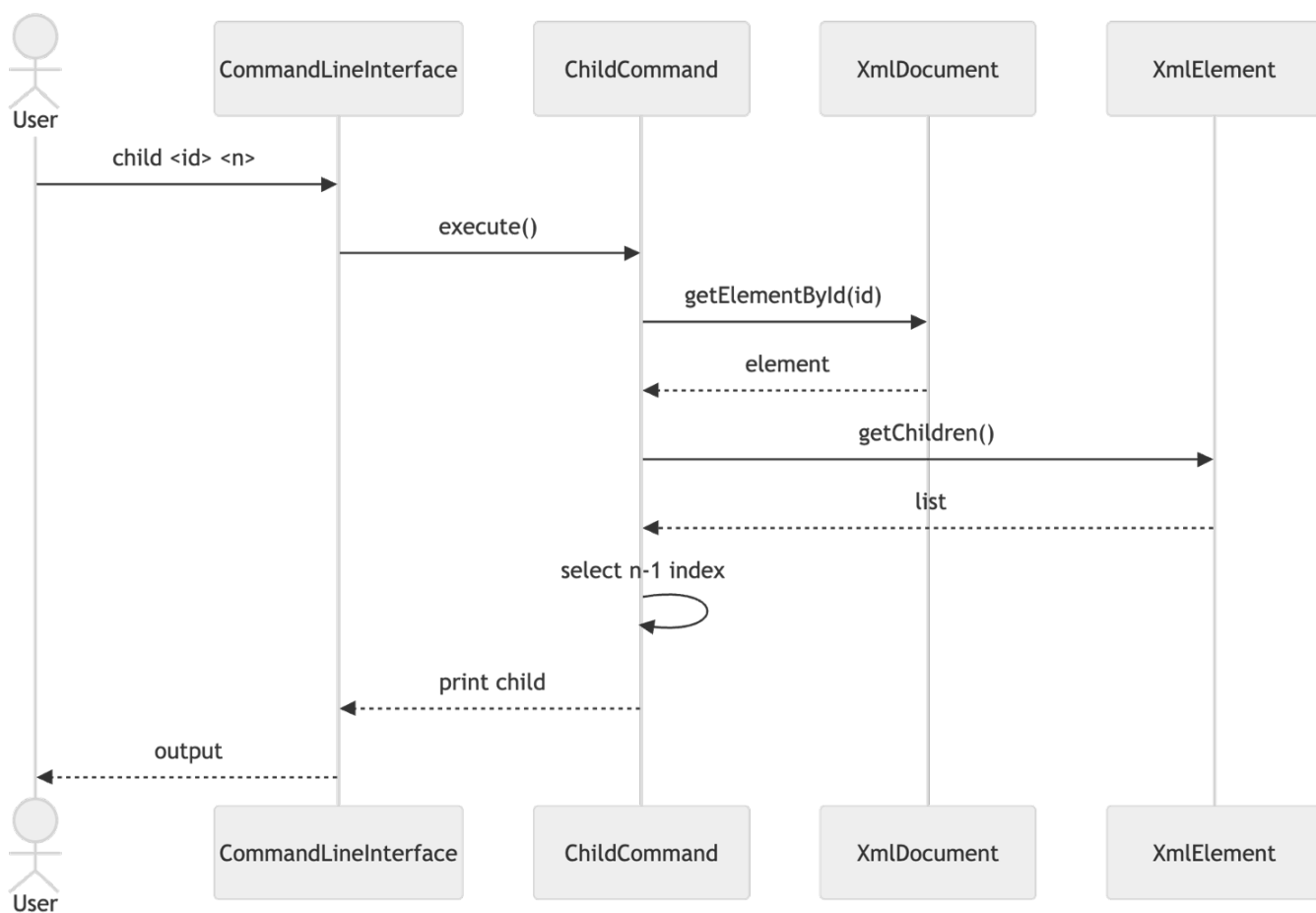
Диаграма 2: XPath заявка



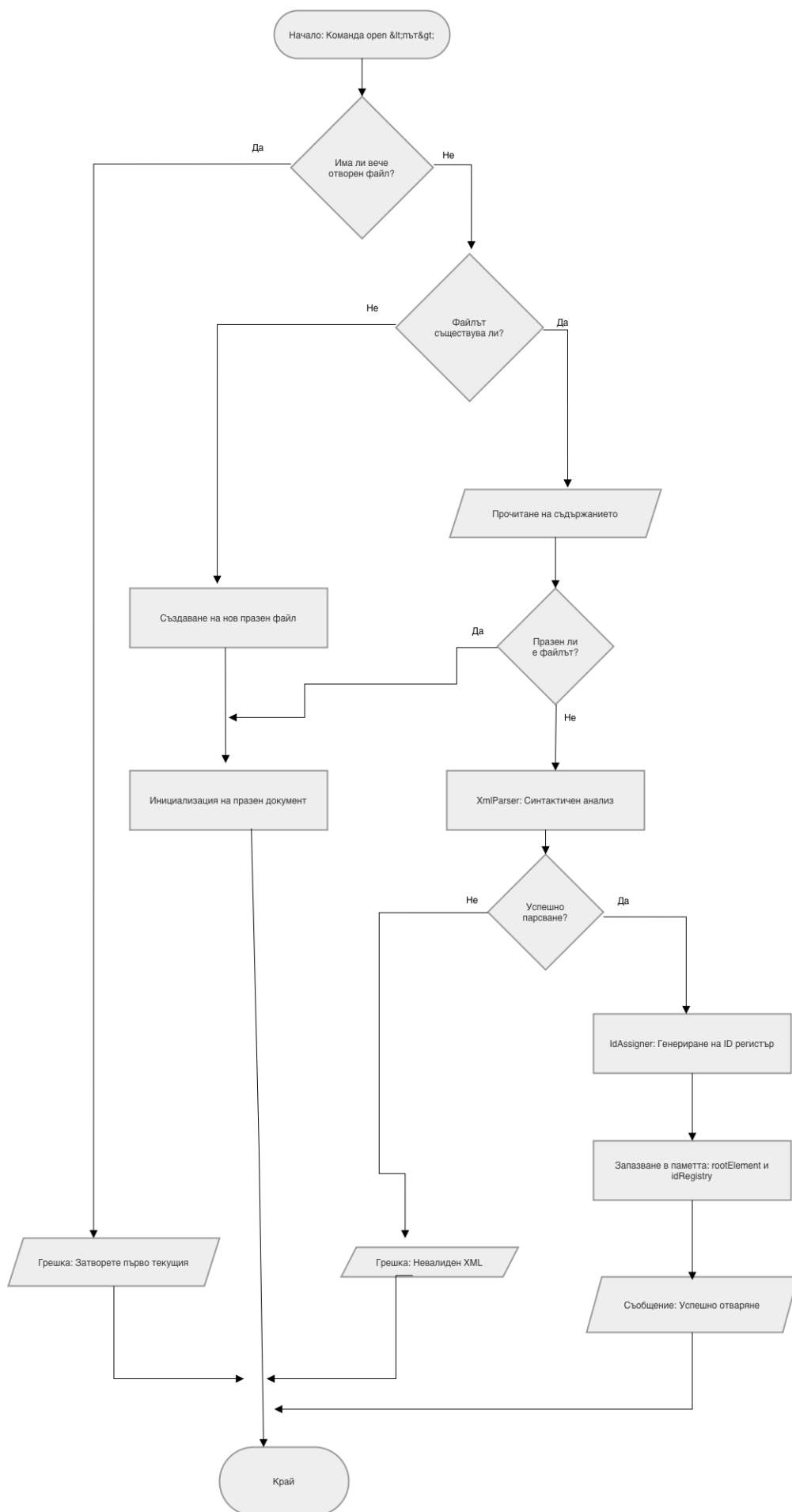
Фигура 6: Блок-схема на *IdAssigner*



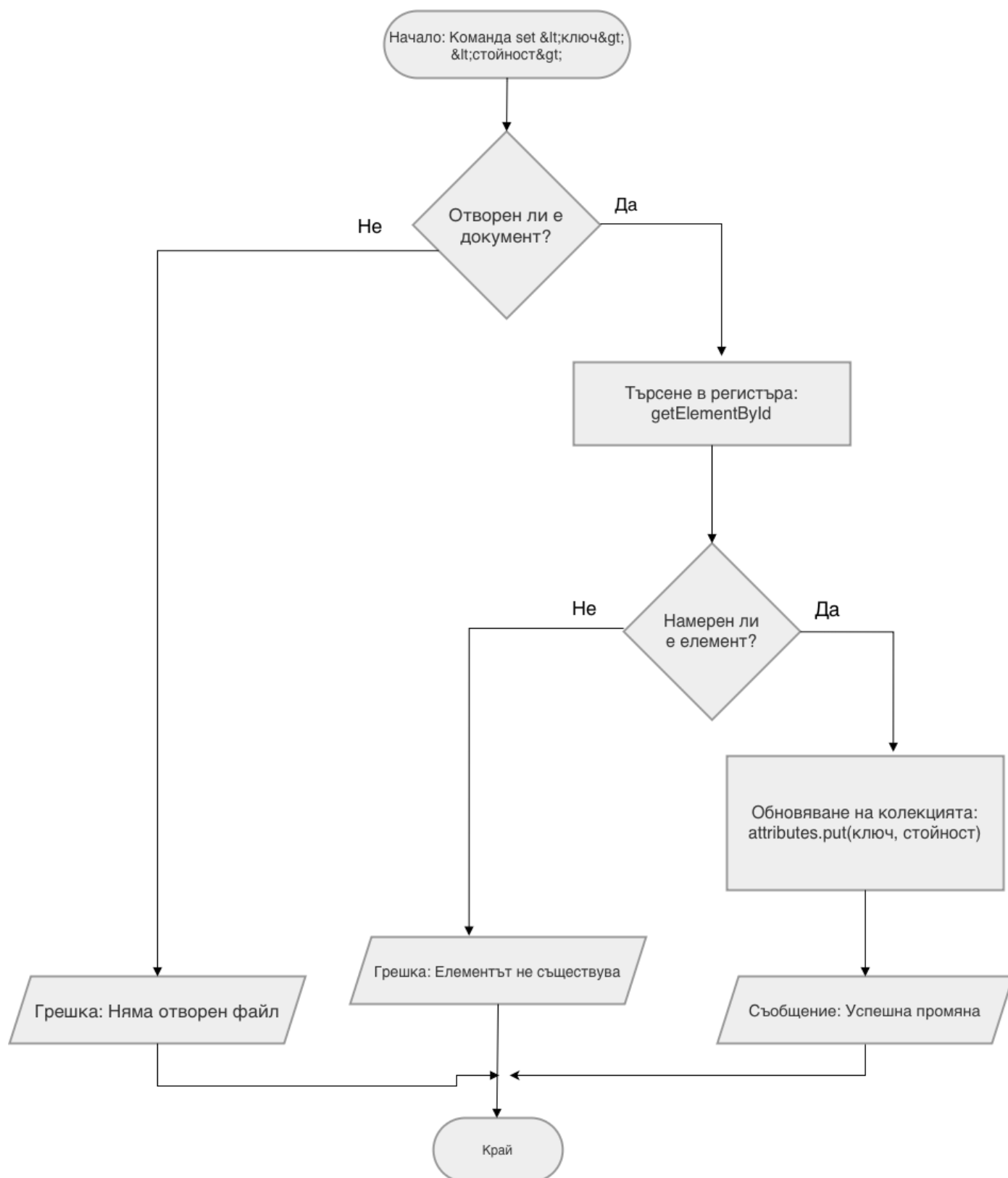
Фигура 7: Схема на AxisResolver



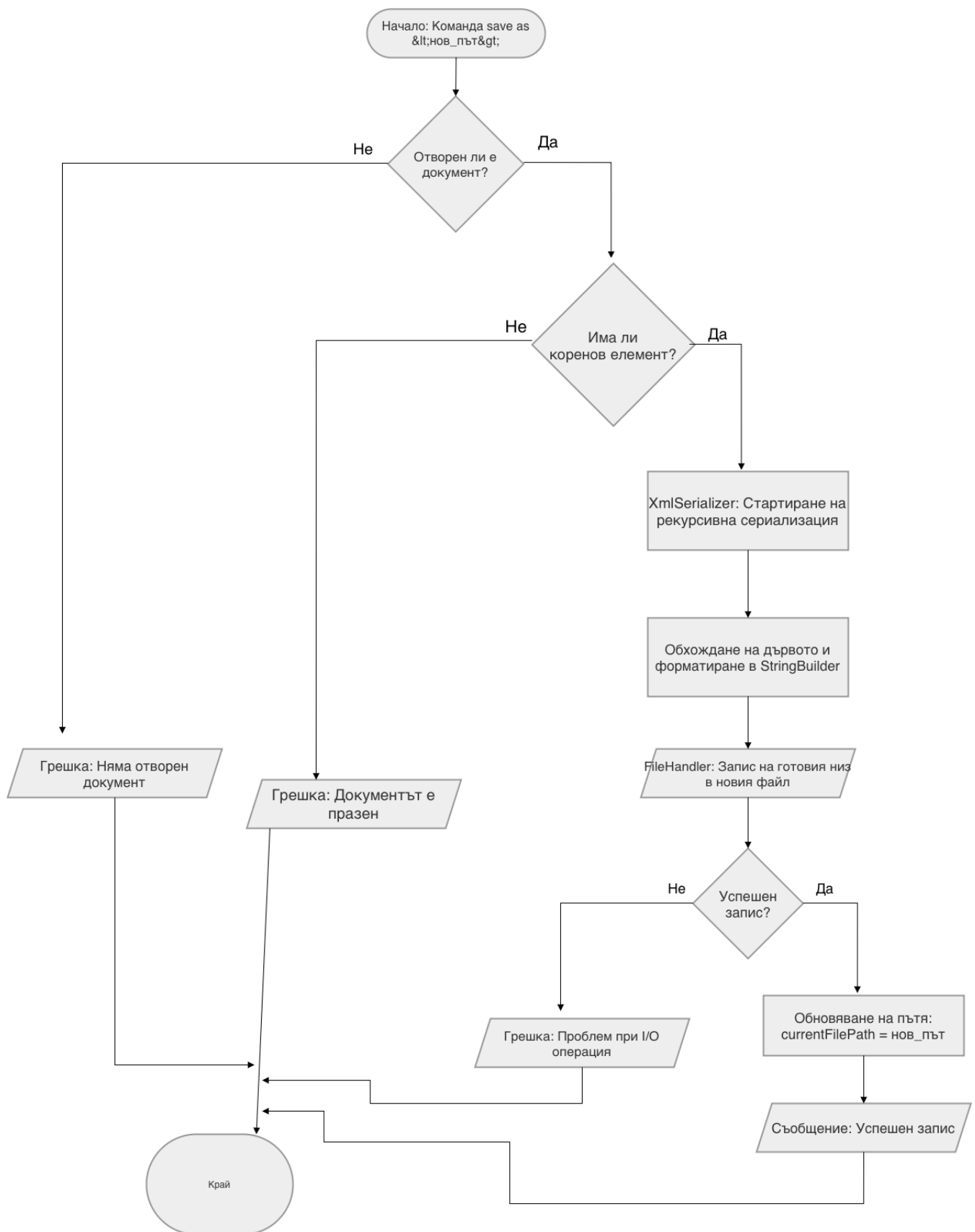
Диаграма 3: Child команда



Фигура 8: Блок-схема на командата open



Фигура 9: Блок-схема на командата set



Фигура 10: Блок-схема на командата *save as*

Глава 4. Реализация

4.1 Реализация на класове

Програмната реализация на курсовия проект е осъществена на обектно-ориентирания език Java. По време на разработката са приложени най-добрите съвременни индустриални практики за чисто писане на код (*Clean Code*), строга капсулация на данните, полиморфизъм и конвенции за именуване. За да се избегне създаването на монолитен и труден за поддръжка софтуер, архитектурата е изградена на стриктен модулнен принцип, следвайки концепцията за разделяне на отговорностите (*Separation of Concerns*). Системата е разделена на шест специализирани пакета. В тази секция всички разработени класове са организирани в основни функционални групи, като са анализирани техният интерфейс, вътрешно състояние и роля в цялостната архитектура.

Група 1: Информационен модел на данните (Paket models)

Тази група дефинира фундаменталните структури от данни, отговарящи за изграждането и управлението на абстрактното синтактично дърво в оперативната памет на системата.

• **Клас `XmlElement` (Реализация на дървовидната структура и *Namespaces*):** Сърцевината на модела е класът `XmlElement`. Той служи като основен градивен блок. Пази пълното си състояние (таг, текстово съдържание, речник от атрибути и списък от деца) и предлага методи за защитен достъп и модификация. Важно архитектурно решение е използването на структурата `LinkedHashMap` за съхранение на атрибутите, което гарантира запазването на точния ред на вмъкване при последваща сериализация. За разлика от плоските масиви, тук е реализирана същинска дървовидна йерархия чрез двупосочна свързаност. Методът `addChild` автоматично задава родителя на добавеното дете (`child.setParent(this)`), което е критично за правилното изкачване по дървото (навигация по XPath осите `parent` и `ancestor`). В класа е заложено и дефанзивно програмиране (*Defensive Programming*) – конструкторът и методите за добавяне на атрибути стриктно валидират входните данни, хвърляйки `IllegalArgumentException` при опит за създаване на таг с празно име или атрибут с `null` стойност. Добавена е и функционалност за динамично резолвиране и интелигентно игнориране на XML пространства от имена (*Namespaces*) чрез методите `getLocalName()` и `getNamespacePrefix()`. За да се избегне архитектурно претрупване и свръхпроектиране (*overengineering*), логиката за обработка на пространства от имена умишлено не е изнесена в отделен помощен клас. Тъй като тази функционалност зависи изцяло от вътрешното състояние на възела (неговия таг, неговите атрибути и референцията към родителя), е приложен подходът *Rich Domain Model* (Богат домейн модел). По този начин всеки обект от тип `XmlElement` сам знае как да управлява и резолвира собствените си свойства, което демонстрира стриктно спазване на принципа за капсулация в обектно-ориентираното програмиране (ООП).

Допълнително е предефиниран методът `toString()`, който връща красиво системно обобщение на обекта, използвано за извеждане на резултати в конзолата.

```
// Извадка от XmlElement.java: Дефанзивно програмиране и управление на дървото
public XmlElement(String tag) {
    if (tag == null || tag.trim().isEmpty()) {
        throw new IllegalArgumentException("Tag name cannot be null or empty.");
    }
    this.tag = tag;
    this.textContent = "";
    this.attributes = new LinkedHashMap<>();
    this.children = new ArrayList<>();
    this.parent = null;
}

public void addChild(XmlElement child) {
    child.setParent(this); // Изграждане на двупосочна връзка
    this.children.add(child);
}

// Рекурсивно резолвиране на XML пространства от имена (Namespaces)
public String getNamespaceURI(String prefix) {
    String attrKey = (prefix == null || prefix.isEmpty()) ? "xmlns" : "xmlns:" +
prefix;
    String uri = getAttributeByKey(attrKey);
    if (uri != null) {
        return uri;
    } else if (parent != null) {
        return parent.getNamespaceURI(prefix); // Рекурсивно търсене нагоре по
дървото
    }
    return null;
}

// Предефиниран метод за нуждите на конзолния интерфейс
@Override
public String toString() {
    return "XmlElement { tag = '" + tag + "', id = '" + getId() +
        "', attributes count = " + attributes.size() +
        ", children count = " + children.size() + " }";
}
```

• **Клас XmlDocument (Главен контекстен мениджър):** Това е централният оркестратор на приложението. Той капсулира цялостния жизнен цикъл на файла чрез вътрешните променливи `currentFilePath`, `isFileOpened` и `rootElement`. Класът стриктно следва принципа за единствена отговорност (*Single Responsibility Principle - SRP*), като не се опитва да чете или записва данни сам, а делегира тези системни операции на външни инстанции – `FileHandler` (за работа с диска) и `XmlSerializer` (за форматиране на изхода). Изключително важен архитектурен елемент в този клас е методът `open()`. (фиг. 8) Той действа като координатор: извиква входно-изходните операции, предава прочетения стринг на `XmlParser`, улавя и обработва евентуални синтактични грешки (`XmlParseException`) и накрая подава валидираното дърво към `IdAssigner`. Най-важната информационна структура вътре в `XmlDocument` е глобалният регистър `Map<String, XmlElement> idRegistry`. Когато нов файл се зареди успешно чрез метода `open()` (Диаграма 1), регистърът се попълва автоматично чрез алгоритъм с две минавания, осигурявайки константно време за достъп $O(1)$ до всеки един възел в системата чрез метода `getElementById()`. Това елиминира необходимостта от тежки рекурсивни търсения при изпълнение на команди за модификация.

```
// Извадка от XmlDocument.java: Оркестрация на жизнения цикъл и управление на грешки
public void open(String filePath) {
    if (isFileOpened) {
        System.out.println("Error: A file is already opened. Please close it first.");
        return;
    }

    try {
        String content = loadFileContent(filePath);
        // Делегиране на синтактичния анализ
        XmlElement parsedRoot = (content != null) ? parseContent(content) : null;

        // Инициализация на документа и попълване на глобалния регистър
        initDocument(filePath, parsedRoot);
        System.out.println("Successfully opened " + filePath);

    } catch (XmlParseException e) { // прихващане на грешки в йерархията
        System.out.println("XML Error: " + e.getMessage());
    } catch (IOException e) { // Прихващане на системни грешки
        System.out.println("Error: Could not read file " + filePath + ". " + e.getMessage());
    }
}

// Осигуряване на O(1) достъп до обектите в паметта
```

```

public XmlElement getElementById(String id) {
    if (idRegistry != null) {
        return idRegistry.get(id);
    }
    return null;
}

```

Група 2: Команден интерфейс и операции (Пакет commands)

Този пакет реализира цялостния потребителски интерфейс тип команден ред (*Command Line Interface - CLI*) чрез прилагане на архитектурния шаблон за дизайн *Command Pattern* (Команда). Този избор гарантира пълно декупиране (отделяне) на логиката по приемане на потребителския вход от същинската бизнес логика на приложението. Системата става лесна за разширяване, спазвайки принципа *Open/Closed Principle* – добавянето на нова функционалност изисква просто създаване на нов команден клас, без промяна в ядрото на интерфейса. (фиг. 2)

- **Клас *CommandLineInterface*:** Този клас действа като инициатор (*Invoker*) на командите. Той капсулира в себе си основния жизнен цикъл на програмата чрез безкраен цикъл за четене от стандартния вход посредством обекта *Scanner*. Изключително важен аспект от реализацията на този клас е методът *parseArguments()*. За разлика от стандартното разделяне по интервали, този метод имплементира специализирана логика за обработка на пътища към файлове. Например, при командата *save as* (фиг. 10), той разпознава, че всичко след ключовата дума *as* трябва да се третира като единен аргумент (път), дори ако съдържа интервали. Това решава често срещан проблем при работа с файлови системи. Класът притежава и защитен механизъм *requiresOpenDocument()*. Той реализира софтуерна архитектура от тип *Fail-Fast* (бързо прекъсване), която проверява състоянието на *XmlDocument* в паметта преди всяка операция. Ако потребителят се опита да изтрие възел или да изпълни *XPath* заявка без зареден файл, системата прекратява изпълнението моментално с информативно съобщение, предотвратявайки критични грешки (*NullPointerExceptions*).

- **Абстрактен клас *Command*:** Дефинира общата рамка за всяка потребителска инструкция. Той съхранява метаданни за командата: уникално име, синтактична структура (*argsSyntax*) и подробно описание за помощното меню (*help*). Чрез дефинирането на абстрактния метод *execute()*, класът налага полиморфен договор – всяка конкретна имплементация е длъжна да реализира собствена логика за изпълнение, като същевременно използва споделените методи за достъп до аргументите *getArgs()*.

- **Конкретни командни класове (*ChildCommand*, *XPathCommand*, *SetCommand* (фиг. 9) и др.):** Всяка команда е реализирана като автономен обект. (Таблица 2)

Класът *ChildCommand* демонстрира висока устойчивост на грешки чрез защитно парсане на числови аргументи. Използването на блок *try-catch* при преобразуването на низове към цели числа (*Integer.parseInt()*) предпазва програмата от срив при некоректен потребителски вход. Освен това, класът реализира трансформация от

човешко към машинно броене (индекс $n-1$), осигурявайки плавно потребителско изживяване. (Диаграма 3)

Класът **XpathCommand** служи като мост между командния интерфейс и изчислителния модул. Той използва принципа на инжектиране на зависимости (*Dependency Injection*), като приема инстанции на `XmlDocument` и `XPathService`. Командата извлича стартовия възел чрез $O(1)$ търсене в регистъра и предава управлението на XPath енджина, след което итерира през резултатите и ги извежда в структуриран вид.

```
// Извадка от CommandLineInterface.java: Интелигентна обработка на аргументи
private String[] parseArguments(String input) {
    // Разделяне на максимум 4 части за общия случай (важно за командата set)
    String[] args = input.split("\\s+", 4);
    String commandName = args[0].toLowerCase();

    // Ако командата е open, вземаме всичко след името на командата като един
    аргумент
    if (commandName.equals("open")) {
        String[] parts = input.split("\\s+", 2);
        if (parts.length > 1) {
            return new String[]{"open", parts[1]};
        }
    }

    // Ако командата е save as, вземаме пътя като един аргумент (може да съдържа
    интервали)
    if (commandName.equals("save") && args.length > 1 &&
        args[1].equalsIgnoreCase("as")) {
        String[] parts = input.split("\\s+", 3);
        if (parts.length >= 3) {
            return new String[]{"save", "as", parts[2]};
        }
    }

    return args;
}
```

Команда	Аргументи	Описание
<code>open</code>	<code><file_path></code>	Зарежда XML документ от файловата система. Ако файлът не съществува, създава нов.
<code>save</code>	-	Записва текущите промени обратно в оригиналния файл.
<code>save as</code>	<code><file_path></code>	Експортира текущото състояние на документа в нов файл по зададен път.
<code>print</code>	-	Визуализира форматираната XML структура в конзолата.
<code>set</code>	<code><id> <key> <value></code>	Променя стойност на атрибут или съдържание на възел по уникален идентификатор.
<code>xpath</code>	<code><id> <query></code>	Изпълнява XPath заявка, започвайки от възела със съответното ID.
<code>exit</code>	-	Прекратява изпълнението на програмата.

Таблица 2. Описание на команди в CLI

```
// Извадка от XPathCommand.java: Интеграция между модулите
@Override
public void execute() {
    if (getArgs().length < 3) {
        System.out.println("Usage: xpath <id> <XPath>");
        return; }
    String startId = getArgs()[1];
    String path = getArgs()[2];
    // Извличане на контекстния възел от регистъра на документа
    XmlElement startNode = document.getElementById(startId);
    if (startNode == null) {
        System.out.println("Error: Element with ID '" + startId + "' not found.");
        return;
    }
    // Изпълнение на заявката чрез XPath сървиса
    List<String> results = xpathService.evaluate(startNode, path);
    if (results.isEmpty()) {
```

```

        System.out.println("No matches found.");
    } else {
        // Отпечатване на филтрираните резултати
        for (String value : results) {
            System.out.println(value);
        }
    }
}
}

```

Група 3: Синтактичен анализ и валидация (Пакет `parsers` и `exceptions`)

Тази функционална група е отговорна за трансформирането на плоския, неструктуриран текстов поток от входния файл в строга, йерархично свързана поредица от обекти в оперативната памет. Реализацията е проектирана чрез архитектурния шаблон *Strategy* (Стратегия), което позволява гъвкаво превключване между различни алгоритми за синтактичен анализ в зависимост от сложността на конкретния XML таг. Системата е изградена изцяло със собствени софтуерни инструменти, без използването на външни библиотеки за обработка на XML (като DOM или SAX), което гарантира ниско ниво на контрол върху паметта и бързодействие.

- **Интерфейс `ElementParser`:** Дефинира общия полиморфен договор чрез метода `parse(String content)`. Този интерфейс позволява на главния оркестратор да делегира обработката на низовете към специализирани компоненти, без да се интересува от техния вътрешен алгоритъм. Методът е проектиран да хвърля проверимото изключение `XmlParseException` при откриване на синтактични аномалии.

- **Клас `SimpleTagParser`:** Специализирана стратегия за бързо обработване на елементарни тагове, които не съдържат вътрешни атрибути (напр. `<title>`). Алгоритъмът тук е оптимизиран за скорост – той директно извлича името на тага чрез премахване на празни пространства, избягвайки сложни логически проверки, което подобрява общата производителност при обработка на големи текстови масиви.

- **Клас `AttributeTagParser`:** Тази стратегия е софтуерно по-комплексна и се задейства автоматично при наличие на тагове с вградени данни (напр. `<book id="b1" status="ok">`). Изключително важно архитектурно решение тук е пълното отхвърляне на бавните регулярни изрази (*Regular Expressions* - *Regex*). Вместо тях е имплементиран високоефективен алгоритъм за итеративна манипулация на низове чрез методите `indexOf('=')` и `substring()`. Алгоритъмът сканира съдържанието линейно, изолирайки ключовете и стойностите между кавичките. Този подход осигурява софтуерна защита срещу често срещани грешки в XML синтаксиса и работи със значително по-ниска консумация на изчислителен ресурс.

- **Клас `xmlParser` (Главен оркестратор):** Този клас управлява цялостния процес на синтактичен анализ. Той итерира през XML текста чрез цикъл `while`, прецизно локализирайки индексите на отварящите и затварящите скоби. За да се изгради

правилната дървовидна йерархия, е използвана фундаменталната структура от данни *Stack* (Стек) от обекти тип `XmlElement`.

- При откриване на отварящ таг, алгоритъмът създава нов обект и го поставя на върха на стека (*push*).

- При срещане на затварящ таг, се извиква методът `handleClosingTag()`, който извършва критична валидация за съвпадение между таговете. Ако имената не съответстват, се хвърля персонализираното изключение `XmlParseException`. Този механизъм гарантира, че в паметта ще бъде заредено единствено структурно правилно и балансирано дърво.

- **Клас `IdAssigner` (Реализация на уникални идентификатори):** Този компонент гарантира софтуерната цялост на глобалния регистър на документа. Тъй като XML стандартът изисква атрибутът `id` да бъде уникален, а потребителските файлове често съдържат дубликати или липсващи идентификатори, е разработен алгоритъм с две минавания (*Two-pass algorithm*). (фиг. 6)

1. *Първо минаване:* Рекурсивно обхождане на дървото и преброяване на честотата на всяка стойност на `id` чрез Хеш-карта (`Map`).

2. *Второ минаване:* Динамична корекция на колизиите. Ако дадено ID се повтаря, системата инжектира суфикс (напр. `id_1`), а при пълна липса на идентификатор – генерира нов маркер по схемата `gen-1`. По този начин се гарантира, че всяка конзолна команда ще има надежден и уникален път до всеки възел в паметта.

```
// Извадка от XmlParser.java: Управление на йерархията чрез Стек (Stack)
private void handleOpeningTag(String tagContent) throws XmlParseException {
    boolean isSelfClosing = false;

    if (tagContent.endsWith("/")) {
        isSelfClosing = true;
        tagContent = tagContent.substring(0, tagContent.length() - 1).trim();
    }

    // Динамичен избор на стратегия (Strategy Pattern) въз основа на съдържанието
    ElementParser parser = tagContent.contains(" ") ? attributeTagParser :
    simpleTagParser;
    XmlElement newElement = parser.parse(tagContent);

    if (root == null) {
        root = newElement; // Определяне на корена на документа
    } else if (!stack.isEmpty()) {
        stack.peek().addChild(newElement); // Автоматично задаване на родител
    }

    if (!isSelfClosing) {
```

```

        stack.push(newElement); // Проследяване на дълбочината на влягане
    }
}

```

- **Пакет exceptions (Клас XmlParseException):** Този клас реализира персонализирано, проверимо изключение, което наследява базовия клас Exception. Неговата роля е да капсулира специфичните съобщения за грешка, възникнали по време на синтактичния анализ. Обособяването му в отделен пакет гарантира, че системата може да разпознава и обработва грациозно структурни аномалии във входните данни, без да се прекратява жизненият цикъл на приложението.

Група 4: Енджин за XPath заявки (Пакет xpath)

Тази група представлява най-сложния изчислителен и аналитичен модул в софтуерната система. Неговата роля е да преобразува текстови низове, съдържащи XPath заявки (напр. `section/book[1]/title`), в динамично филтрирани списъци от обекти. Реализацията съчетава два мощни шаблона за дизайн: *Chain of Responsibility* (Верига от отговорности) и *Strategy* (Стратегия). Тази комбинация позволява заявката да бъде обработена стъпка по стъпка, като всеки фрагмент от пътя се анализира от специализиран компонент. (Диаграма 2)

- **Клас xpathService (Централен процесор):** Този клас е входната точка за всяка заявка за търсене. Неговата логика е организирана около итеративен процес на филтриране. Когато бъде подадена заявка, тя се разделя на отделни стъпки чрез символа за наклонена черта (/). Процесорът поддържа междинен списък от резултати, който първоначално съдържа само контекстния (стартовия) възел. За всяка следваща стъпка, xpathService обхожда регистрираните оператори и чрез метода `canHandle()` идентифицира правилната стратегия за филтриране. Процесът приключва, когато всички стъпки са изпълнени или когато списъкът с резултати стане празен.

- **Абстрактен клас xpathOperator:** Дефинира общата рамка за всички филтриращи операции. Той съдържа критичния защитен метод `resolveAxisAndTag()`, който се споделя от всички наследници. Този метод е отговорен за отделянето на XPath оста (напр. `ancestor::`) от името на търсения таг, преди да се приложи специфичното за оператора филтриране (по индекс или атрибут). По този начин се гарантира преизползваемост на кода и спазване на принципа за суха разработка (*Don't Repeat Yourself - DRY*).

- **Клас tagNavigationOperator:** Това е основната стратегия за навигация, която се задейства при стъпки, съдържащи само име на таг. Операторът извлича наследниците на текущите възли и филтрира тези, които съответстват на зададения критерий (име или универсален оператор *).

- **Клас indexOperator:** Реализира специфичен синтаксис за избор на елемент по неговата позиция (в квадратни скоби [`n`]). Тъй като в стандарта XPath индексите са базирани на единица, а вътрешните структури на Java използват нулев индекс, операторът

извършва математическа корекция, извличайки точно определения елемент от текущия списък.

- **Клас `AttributeFilterOperator`:** Имплементира най-сложната форма на филтриране – по стойност на атрибут или съдържание на вложен таг (в кръгли скоби (`key=value`)). Алгоритъмът анализира съдържанието на скобите, разделя ключа от стойността чрез метода `split()` и извършва интелигентно почистване на кавичките. След това се извършва линейна проверка на всеки елемент от текущия списък, за да се потвърди наличието на съответния атрибут.

- **Клас `AttributeAccessor`:** Този компонент реализира така наречената терминална (крайна) операция, разпознавайки символа за достъп до атрибут (`@`). За разлика от останалите оператори, той прекратява веригата от обхождане на XML възли и връща списък от крайни стойности тип `String`. Това позволява на потребителя да извлича директно данни, а не цели обекти.

- **Клас `AxisResolver` (Помощен клас за обхождане на оси):** Този клас е реализиран като помощен (*Utility class*) със статични методи, което елиминира необходимостта от създаване на нови обекти в паметта при всяка заявка, подобрявайки производителността. Той имплементира пълната логика за движение по софтуерното дърво чрез `switch` конструкция, обхващаща всички поддържани оси: `child` (директни деца), `parent` (директен родител), `ancestor` (всички прародители), `descendant` (всички наследници в дълбочина) и техните вариации, включващи текущия възел (*-or-self*).

```
// Извадка от AxisResolver.java: Алгоритми за навигация по осите на дървото
public static List<XmlElement> resolve(List<XmlElement> contextNodes, String
stepBase) {
    String axis = "child"; // Ос по подразбиране
    String nodeTest = stepBase;

    // Разпознаване на синтаксис за ос (axis::tag)
    if (stepBase.contains("::")) {
        String[] parts = stepBase.split("::", 2);
        axis = parts[0].trim();
        nodeTest = parts[1].trim();
    }

    List<XmlElement> result = new ArrayList<>();
    for (XmlElement node : contextNodes) {
        // Извличане на всички възли по съответната ос
        List<XmlElement> axisNodes = getAxisNodes(node, axis);
        for (XmlElement axisNode : axisNodes) {
            // Филтриране чрез интелигентно съвпадение (включително Namespaces)
            if (matchesNodeTest(axisNode, nodeTest)) {
                if (!result.contains(axisNode)) {
                    result.add(axisNode); // Предотвратяване на дублиране на възли
                }
            }
        }
    }
}
```

```

        }
    }
}
return result;
}
// Извадка от IndexOperator.java: Филтриране на списък по конкретен индекс
@Override
public List<XmlElement> apply(List<XmlElement> parents, String step) {
    String tagName = step.substring(0, step.indexOf("[")).trim();
    String content = step.substring(step.indexOf("[") + 1,
step.indexOf("]").trim());
    int index;
    try {
        index = Integer.parseInt(content); // Безопасно преобразуване на индекса
    } catch (NumberFormatException e) {
        return new ArrayList<>();
    }
    // Първоначална навигация по ос и таг, последвана от селекция по индекс
    List<XmlElement> matches = resolveAxisAndTag(parents, tagName);
    return selectByIndex(matches, index); // Делегиране към помощен метод
}

```

Група 5: Входно-изходни операции и сериализация (Пакет io)

Тази функционална група е проектирана да декуплира (отдели) изцяло работата с файловата система на операционната система от основната бизнес логика на приложението. Чрез това архитектурно изолиране на системните процеси се постига висока степен на модулност – ако в бъдеще се наложи промяна в начина на съхранение (например преминаване към облачна структура или база данни), това ще изисква промени единствено в този пакет, без да се засягат останалите части от софтуера. (фиг. 1)

- **Клас `FileHandler`:** Този компонент капсулира всички входно-изходни операции (*Input/Output operations - I/O*) на ниско ниво. За реализацията му са избрани съвременните библиотеки от пакета *Java NIO (New Input/Output)*, като се използват методите `Files.readString()`, `Files.createFile()` и класа `Path`. Този подход гарантира по-бърза и сигурна работа с потоците от данни в сравнение със старите методи за четене. Класът имплементира дефанзивна логика – преди всеки опит за четене се извършва проверка за съществуване на файла чрез `Files.exists()`. В случай че потребителят се опита да отвори несъществуващ път, програмата не прекратява работа с грешка, а автоматично създава нов, празен файл, подготвяйки системата за работа. Това осигурява плавно потребителско изживяване и софтуерна устойчивост.

• **Клас `XmlSerializer`:** Докато `FileHandler` се грижи за суровия текст, `XmlSerializer` е отговорен за интелигентното преобразуване на йерархичното дърво от паметта обратно в строго структуриран и валиден XML формат. Той имплементира сложен рекурсивен алгоритъм за сериализация, който обхожда всеки възел и неговите наследници в дълбочина. Изключително важна функционалност е ролята на класа като форматиращ инструмент (*Pretty-printer*). Системата автоматично изчислява и прилага отместване от 4 интервала за всяко ниво на влагане, което прави крайния файл лесен за четене от хора. Друг критичен аспект е методът `escapeXml()`. В XML определени символи (като `<`, `>`, `&`, `"`, `'`) имат специално значение за синтаксиса. Ако те присъстват в текстовото съдържание на тага, те биха „счупили“ структурата на документа. Сериализаторът интелигентно сканира всеки низ и заменя тези символи с техните безопасни XML еквиваленти (сурогати), гарантирайки, че записаният файл ще бъде напълно валиден и съвместим с всеки друг XML четец. Подробният алгоритъм за рекурсивна сериализация и ескейпване е разгледан в секция 4.1.1.

4.1.1. Алгоритми и оптимизации

При разработката на софтуерната архитектура са приложени няколко ключови алгоритмични оптимизации (Таблица 4), гарантиращи висока производителност и устойчивост при работа с големи файлове:

а) Оптимизиран синтактичен анализ чрез Стек и `IndexOf` ($O(n)$ сложност)

За десериализацията на данните не се използват външни библиотеки. Алгоритъмът в `XmlParser` използва структура от данни Стек (*Stack*) за проследяване на йерархията, предпазвайки програмата от критична грешка при препълване на паметта (*StackOverflowError*). Още по-важното е, че при обработката на атрибути в `AttributeTagParser` е напълно избегната употребата на бавни регулярни изрази (*Regex*). Използвани са итеративни низови операции с `indexOf('=')` и `indexOf('\"')`, което ускорява сканирането многократно.

```
// Извадка от XmlParser.java: Управление на стека и строга валидация при затварящ таг
private void handleCloseTag(String tagContent) throws XmlParseException {
    String closingTagName = tagContent.substring(1).trim();

    if (stack.isEmpty()) {
        throw new XmlParseException("Closing tag </" + closingTagName + "> has no
matching opening tag.");
    }
    XmlElement lastOpened = stack.peek();
    // Проверка за съответствие между отварящ и затварящ таг
    if (!lastOpened.getTag().equals(closingTagName)) {
        throw new XmlParseException("Mismatched tags: expected </" +
```

```

        lastOpened.getTag() + "> but found </" + closingTagName +
        ">. Last opened element: " + lastOpened);
    }
    stack.pop(); // Освобождаване на текущото йерархично ниво при успешно съвпадение
}

```

б) Алгоритъм с две минавания (*Two-pass algorithm*) за дедупликация

Използван в класа `IdAssigner`, този алгоритъм гарантира, че няма счупени връзки или колизии в регистъра при зареждане на структурно увредени файлове. При първото минаване се попълва Хеш-карта с честотите на срещане. При второто минаване (`assignUniqueIds`) конфликтите се резолвират чрез интелигентно добавяне на числови суфикси.

```

// Извадка от IdAssigner.java: Резолвиране на дублирани и липсващи идентификатори
private void assignUniqueIds(XmlElement element, Map<String, Integer> idFrequency,
Map<String, Integer> idCounters, Map<String, XmlElement> registry) {
    String rawId = element.getId();

    if (rawId == null || rawId.isEmpty()) {
        // Генериране на ново ID, ако такова липсва (gen-1, gen-2...)
        element.setId("gen-" + generatedIdCounter++);
    } else {
        int freq = idFrequency.getOrDefault(rawId, 1);
        if (freq > 1) {
            // Резолвиране на дублирано ID чрез добавяне на суфикс (брояч)
            int counter = idCounters.getOrDefault(rawId, 0) + 1;
            idCounters.put(rawId, counter);
            element.setId(rawId + "_" + counter);
        }
    }
    // Записване на окончателното съответствие ID -> елемент в регистъра
    registry.put(element.getId(), element);

    for (XmlElement child : element.getChildren()) {
        // Рекурсивно извикване за всички наследници в дървото
        assignUniqueIds(child, idFrequency, idCounters, registry);
    }
}

```

в) Рекурсивна сериализация с управление на паметта (*Memory Management*) и ескейпване

При записването на XML структурата обратно във файл, `XmlSerializer` използва `StringBuilder`. Това представлява мутабилен буфер,

който драстично ускорява сериализацията и щади системата за автоматично почистване на паметта (*Garbage Collector*). Използвана е модулна архитектура – логиката е разделена на помощни методи (`appendAttributes`, `appendContent`). Паралелно с това се прилага алгоритъм за ескейпване на специални символи (`escapeXml`), който гарантира синтактичната валидност на изходния файл.

// Извадка от `XmlSerializer.java`: Рекурсивно изграждане на съдържанието и ескейпване

```
private void appendContent(StringBuilder sb, XmlElement element, int indentLevel,
String spaces) {
```

```
    String text = element.getTextContent();
    if (text != null && !text.isEmpty()) {
        // Защита на специалните символи преди запис
        sb.append(escapeXml(text));
    } else if (!element.getChildren().isEmpty()) {
        sb.append("\n");
        for (XmlElement child : element.getChildren()) {
            // Рекурсивно извикване на сериализацията за всяко дете (дълбоко
            обхождане)
            sb.append(serialize(child, indentLevel + 1));
        }
        sb.append(spaces); // Връщане на отместването за затварящия таг
    }
}
```

// Извадка от `XmlSerializer.java`: Методи за сигурност на XML синтаксиса

```
private String escapeXml(String input) {
    if (input == null) return "";
    return input.replace("&", "&amp;")
        .replace("<", "&lt;")
        .replace(">", "&gt;")
        .replace("\"", "&quot;")
        .replace("'", "&apos;");
}
```

d) Оптимизация на търсенето до $O(1)$ чрез *Registry Pattern*

Търсенето по `id` атрибут, реализирано в класа `XmlDocument` чрез структурата `Map<String, XmlElement>`, свежда алгоритмичната сложност от $O(n)$ (при обикновено обхождане в дълбочина) до директен достъп до паметта с константно време $O(1)$. Това е критично за бързодействието на командите за модификация.

// Извадка от `XmlDocument.java`: $O(1)$ достъп до елементите в паметта се за осигуряване на бърз достъп вместо директно обхождане.

```
public void registerElement(String id, XmlElement element) {
```

```

        if (idRegistry != null) {
            idRegistry.put(id, element);
        }
    }
    public XmlElement getElementById(String id) {
        if (idRegistry != null) {
            return idRegistry.get(id);
        }
        return null;
    }
}

```

е) Интелигентно обхождане на XPath осите

Алгоритмите в `AxisResolver` (фиг. 7) дефинират поддръжка за масив от оси. Навигацията нагоре (`parent`, `ancestor` и `ancestor-or-self`) се извършва изцяло итеративно чрез `while` цикъл по референциите, което гарантира линейно време $O(h)$, където h е височината на дървото. Енджинът поддържа филтриране по локално име чрез метода `matchesNodeTest`. (Таблица 3)

Ос (Axis)	Поведение и обхват на търсенето
<code>child::</code>	Извлича преките наследници на текущия възел (това е оста по подразбиране).
<code>parent::</code>	Намира прекия родител на контекстния елемент.
<code>ancestor::</code>	Рекурсивно обхождане на всички нива нагоре до корена (родител, прародител и т.н.).
<code>descendant::</code>	Обхожда всички нива надолу, извличайки цялото поддърво на елемента.
<code>self::</code>	Връща само текущия контекстен възел.
<code>*</code> (Wildcard)	Универсален оператор, който съвпада с всеки таг, независимо от името му.

Таблица 3. Поддържани XPath оси и синтаксис

```

// Извадка от AxisResolver.java: Интелигентно съвпадение на тагове и универсален оператор (*)
private static boolean matchesNodeTest(XmlElement node, String nodeTest) {

```

```

        if ("*".equals(nodeTest)) {
            return true;
        }
        // Проверка за съвпадение с пълното име на тага или само с локалното име (без
        префикс)
        return node.getTag().equals(nodeTest) || node.getLocalName().equals(nodeTest);
    }

    // Извадка от AxisResolver.java: Итеративно изкачване по оста "ancestor"
    private static void collectAncestors(XmlElement node, List<XmlElement> result) {
        XmlElement curr = node.getParent();
        // Итеративно изкачване нагоре по референциите към родителите
        while (curr != null) {
            result.add(curr);
            curr = curr.getParent();
        }
    }
}

```

f) Динамично избиране на оператори (*Strategy Pattern* и *Chain of Responsibility*)

Модулят `XPathService` координира заявките чрез полиморфно прилагане на филтри. Вместо сложни и уязвими масиви от `if-else` проверки, се обхожда колекция от оператори (`IndexOperator`, `AttributeFilterOperator`). Всеки оператор дефинира метод `canHandle()`, чрез който динамично разпознава дали специфичният синтаксис (напр. кръгли или квадратни скоби) е предназначен за него.

```

// Извадка от XPathService.java: Динамичен избор на оператор (Strategy Pattern)
private List<XmlElement> applyOperator(List<XmlElement> elements, String step) {
    for (XPathOperator operator : operators) {
        // Проверка дали конкретният оператор може да обработи текущата стъпка
        if (operator.canHandle(step)) {
            return operator.apply(elements, step);
        }
    }
    // Връщане на празен списък, ако никой оператор не разпознае синтаксиса
    return new ArrayList<>();
}

```

g) Защита от тип *Fail-Fast* и безопасност на типовете

В класа `CommandLineInterface` проверките за системна валидност се извършват приоритетно (*Fail-Fast* подход). Методът `requiresOpenDocument()` проверява състоянието на паметта преди инстанцирането на операционната логика. В самите

команди се извършва безопасно преобразуване чрез блок `try-catch` на подадените индекси, осигурявайки стабилна работа без срилове на виртуалната машина.

```
// Извадка от CommandLineInterface.java: Защита на жизнения цикъл (Fail-Fast подход)
if (requiresOpenDocument(commandName) && !document.isOpened()) {
    // Проверка дали командата изисква отворен файл и дали такъв е зареден в момента
    System.out.println("Error: No file is currently opened.");
    continue; // Прекратяване на текущата итерация на цикъла
}
```

Оптимизация	Реализация	Полза
Registry Pattern	<code>Map<String, XmlElement></code>	O(1) достъп
Stack Parsing	<code>Stack<XmlElement></code>	Безопасна обработка на дълбоки структури
StringBuilder	Сериализация	Намалена консумация на памет
Без Regex	<code>indexOf()</code> и <code>substring()</code>	По-висока производителност
Fail-Fast проверки	CLI валидация	Предотвратяване на runtime грешки
Immutable wrappers	<code>Collections.unmodifiableList()</code>	Защита на вътрешното състояние

Таблица 4: Оптимизации в системата

Глава 5. Тестване

5.1. Планиране и методология на тестването

За да се гарантира надеждността, стабилността и коректността на разработения XML парсър, е проведен задълбочен процес на софтуерно тестване. Избраната методология е **Тестване на черна кутия** (*Black-box testing*), при която системата се тества чрез нейния конзолен интерфейс, без да се анализира директно изпълнението на вътрешния код по време на теста.

Проведени са следните видове тестове:

- а. Функционално тестване (*Functional Testing*): Проверка дали всяка команда отговаря на изискванията в заданието.

б. Тестване за регресия (*Regression Testing*): След добавянето на сложни функционалности като XPath оси и Namespaces, старите базови команди са тествани отново, за да се гарантира, че новата логика не е нарушила тяхната работа.

с. Стрес тестване и валидация (*Edge-case Testing*): Въвеждане на невалидни данни (грешни идентификатори, несъществуващи индекси, опити за модификация на затворен файл), за да се провери дали програмата извежда коректни съобщения за грешка (Error handling), вместо да прекъсва работа неочаквано (*Crash*).

5.2. Описание на тестови сценарии и примери за вход и изход

За целите на цялостното софтуерно тестване е проектиран единен, масивен структурен XML файл с име `library_test.xml`. Той съдържа комплексни елементи, декларации на пространства от имена (*Namespaces*), префикси, както и умишлено заложили структурни аномалии (дублиран идентификатор и липсващ такъв), за да се тества пълният капацитет на подсистемите за валидация още при зареждането на документа.

Код на тестов файл `library_test.xml`:

```
<library                                xmlns:bk="http://library.org/books"
xmlns:auth="http://library.org/authors" id="lib-1" name="Central">
  <section id="sec-1" category="Sci-Fi">
    <bk:book id="b1" bk:status="available">
      <bk:title id="t1">Dune</bk:title>
      <auth:author id="a1">Frank</auth:author>
    </bk:book>
    <bk:book id="b1"> <bk:title id="t2">Foundation</bk:title>
    </bk:book>
    <bk:book> <bk:title id="t3">1984</bk:title>
    </bk:book>
  </section>
</library>
```

5.2.1. Тестове за управление на файловата система и автоматична корекция

Тази група сценарии валидира операциите по десериализация (четене), автоматичното привеждане на модела към изискванията за уникалност и транзакционното запазване на промените на диска.

Сценарий 1.1: Автоматично генериране и дедупликация на уникални идентификатори (Клас IdAssigner)

- Цел: Проверка дали системата успешно идентифицира структурни грешки в ID-тата при отваряне и дали алгоритъмът с две минавания в IdAssigner ги коригира автоматично.
- Входни команди: `open library_test.xml prin`
- Очакван резултат: Конзолата извежда съобщение за успешно отваряне. При последващото изпълнение на командата `print` се вижда, че в оперативната памет йерархията е поправена: липсващото ID е заменено с генериран маркер (`gen-1`), а повтарящите се ID-та на книгите са дедупликирани чрез суфикси:

```
Successfully opened library_test.xml
<library                                xmlns:bk="http://library.org/books"
xmlns:auth="http://library.org/authors" id="lib-1" name="Central">
  <section id="sec-1" category="Sci-Fi">
    <bk:book id="b1_1" bk:status="available">
      <bk:title id="t1">Dune</bk:title>
      <auth:author id="a1">Frank</auth:author>
    </bk:book>
    <bk:book id="b1_2">
      <bk:title id="t2">Foundation</bk:title>
    </bk:book>
    <bk:book id="gen-1">
      <bk:title id="t3">1984</bk:title>
    </bk:book>
  </section>
</library>
```

- Статус на теста: Успешен.

Сценарий 1.2: Отваряне на несъществуващ файл (Създаване на нов)

- Цел: Проверка на защитната логика в FileHandler – ако файлът не бъде намерен на диска, той трябва да се създаде като празен нов документ.
- Входна команда: `open new_database.xml`
- Очакван резултат: Системата създава празен файл в директорията и извежда: `Successfully opened new_database.xml`.
- Статус на теста: Успешен.

Сценарий 1.3: Опит за операция без отворен файл (Fail-Fast защита)

- Цел: Валидация на механизмите в `CommandLineInterface`, предотвратяващи неочаквано прекратяване на програмата (`NullPointerException`) при липса на зареден контекст.
- Предварително условие: Програмата е стартирана наново, но не е извикана команда `open`.
- Входна команда: `save`
- Очакван резултат: Системата прихваща състоянието и отказва изпълнение с любезно съобщение: `Error: No file is currently opened`.
- Статус на теста: Успешен.

Сценарий 1.4: Запазване на промени в нов файл

- Цел: Валидация на сериализатора (`XmlSerializer`) и неговата способност рекурсивно да превърне обектите от паметта обратно в правилно форматиран XML текст.
- Входна команда: `saveas backup.xml`
- Очакван резултат: Извежда се `Successfully saved backup.xml`. На диска се записва нов файл, съдържащ поправените от `IdAssigner` идентификатори и подреден с отстъпи от 4 интервала.
- Статус на теста: Успешен.

5.2.2. Тестове за базова навигация и модификация

Тези тестове проверяват командите за директна манипулация на дървото в оперативната памет чрез коригираните вече уникални идентификатори.

Сценарий 2.1: Извеждане на деца на елемент

- Цел: Валидация на командата `children` и проверка дали тя отразява промените, направени от `IdAssigner`.
- Входна команда: `children sec-1`
- Очакван резултат: Конзолата извежда списък с децата на секцията с техните нови, коригирани в паметта атрибути: `Children of element 'sec-1' (attributes): id="b1_1" bk:status="available" id="b1_2" id="gen-1"`
- Статус на теста: Успешен.

Сценарий 2.2: Вземане на конкретно дете по индекс

- Цел: Валидация на командата `child` и преобразуването на потребителския индекс (базиран на 1) към вътрешния за Java индекс (базиран на 0).
- Входна команда: `child sec-1 1`

- Очакван резултат: Извежда се системно обобщение на първата книга: Child 1 of element 'sec-1':XmlElement { tag = 'bk:book', id = 'b1_1', attributes count = 2, children count = 2 }
- Статус на теста: Успешен.

Сценарий 2.3: Грешен индекс при навигация (Извън границите)

- Цел: Защита от изключението `IndexOutOfBoundsException` при въвеждане на невалиден индекс от потребителя.
- Входна команда: `child sec-1 999`
- Очакван резултат: Системата отпечатва контролирано съобщение: `Error: Element 'sec-1' does not have a child at index 999.`
- Статус на теста: Успешен.

Сценарий 2.4: Модификация и изтриване на атрибути

- Цел: Валидация на командите за редакция на атрибути (`set` и `delete`) върху елементи, чиито идентификатори са били автоматично коригирани със суфикси.
- Входни команди: `set b1_1 year 1965; delete lib-1 name`
- Очакван резултат: `Successfully set attribute 'year' to '1965' for element 'b1_1'.`
`Successfully deleted attribute 'name' from element 'lib-1'.`

Промените се отразяват незабавно в паметта. При последващо изпълнение на командата `print` се вижда, че към първата книга е добавен нов атрибут `year="1965"`, а атрибутът `name` на корена (`lib-1`) е успешно премахнат от структурата.

- Статус на теста: Успешен.

5.2.3. Тестове за стандартни XPath заявки с филтри и индекси

Тези тестове проверяват базовото функциониране на `XPathService`, разпознаването на синтактичните стъпки чрез "Веригата от отговорности" и способността за автоматично пропускане на `namespaces` префиксите при обикновено търсене.

Сценарий 3.1: Навигация надолу по дървото с точни тагове (Без префикси)

- Цел: Проверка на `TagNavigationOperator` и фикса в `XPathOperator`, който позволява намирането на тагове по тяхното локално име, независимо от това дали са записани като `bk:book` или `bk:title`.
- Входна команда: `xpath lib-1 section/book/title`

- Очакван резултат: Извличат се текстовите съдържания на трите заглавия: `Dune Foundation 1984`
- Статус на теста: Успешен.

Сценарий 3.2: Филтриране чрез индекс (*Square brackets*)

- Цел: Валидация на `IndexOperator` и неговия приоритет във веригата.
- Входна команда: `xpath sec-1 book[1]/title`
- Очакван резултат: Извлича се заглавието на втората книга по индекс (Foundation), съобразно zero-based логиката на оператора: `Foundation`
- Статус на теста: Успешен.

Сценарий 3.3: Филтриране по стойност на атрибут (*Round brackets*)

- Цел: Валидация на `AttributeFilterOperator`.
- Входна команда: `xpath lib-1 section(category="Sci-Fi")/book/author`
- Очакван резултат: Система филтрира секцията по атрибут и връща текста на автора в нея: `Frank`
- Статус на теста: Успешен.

Сценарий 3.4: Достъп до стойност на атрибут чрез терминален оператор (`@`)

- Цел: Проверка на класа `AttributeAccessor` за директно извличане на метаданни.
- Входна команда: `xpath sec-1 book(@id)`
- Очакван резултат: Вместо текстово съдържание, операторът прекратява заявката и връща списък с ID-тата на книгите, отразили суфиксите от първия тест: `b1_1 b1_2 gen-1`
- Статус на теста: Успешен.

5.2.4. Тестове за сложни XPath оси (Axes) и Namespaces

Тази последна фаза подлага на стрес тест новия `AxisResolver` и рекурсивните алгоритми за йерархично катерене и спускане, комбинирани с пълна поддръжка на пространства от имена.

Сценарий 4.1: Навигация с прескачане на нива (Descendant Axis)

- Цел: Проверка на алгоритъма за обхождане в дълбочина (DFS) за намиране на елементи навсякъде надолу по дървото.
- Входна команда: `xpath lib-1 descendant::title`
- Очакван резултат: Извличат се заглавията от всички нива без значение в коя книга се намират: `DuneFoundation 1984`
- Статус на теста: Успешен.

Сценарий 4.2: Изкачване нагоре по дървото (Parent / Ancestor Axes)

- Цел: Валидация на двупосочната свързаност на дървото и способността на AxisResolver да се движи нагоре по референциите към parent.
- Входна команда: `xpath t1 parent::book`
- Очакван резултат: Тръгвайки от заглавието `t1`, програмата се качва едно ниво нагоре и връща обекта-родител под формата на системно обобщение (демонстрирайки `b1_1` суфикса и броя на децата): `XmlElement { tag = 'bk:book', id = 'b1_1', attributes count = 2, children count = 2 }`
- Статус на теста: Успешен

Сценарий 4.3: Разпознаване на специфични пространства от имена

- Цел: Проверка дали търсенето работи коректно, когато потребителят реши изрично да потърси с включен префикс или да филтрира по namespace атрибут.
- Входна команда: `xpath lib-1 descendant::book(bk:status="available")/title`
- Очакван резултат: Системата разпознава префикса `bk:`, намира книгата със съответния статус и извежда заглавието ѝ: `Dune`
- Статус на теста: Успешен. Доказва пълната софтуерна съвместимост между AxisResolver, AttributeFilterOperator и модела на пространствата от имена.

Глава 6. Заключение

6.1. Обобщение на изпълнението на началните цели

В рамките на настоящия курсов проект беше успешно проектирана, реализирана и подложена на софтуерно тестване цялостна система за управление, синтактичен анализ (парсване) и обектно-ориентирана манипулация на XML документи. Разработеното конзолно приложение напълно отговаря на заложените в техническото задание функционални и нефункционални изисквания, демонстрирайки стабилност, бързодействие и софтуерна устойчивост.

По време на разработката бяха постигнати следните ключови практически и научно-приложни резултати:

- **Пълна независимост от външни библиотеки:** Синтактичният анализатор бе разработен изцяло "от нулата" (`XmlParser`), разчитайки на алгоритми за линейна текстова обработка и управление на йерархичния контекст чрез структурата от данни стек. Това осигури коректно изграждане на дървовидната йерархия без компромиси с паметта и без използването на вградените в Java DOM или SAX пакети.
- **Трансакционна сигурност и устойчивост на данните (*Fail-Safe*):** Проектът имплементира модел на работа изцяло в оперативната памет (*In-memory computation*),

защитавайки оригиналния файл от повреда. Компонентът `IdAssigner` извършва автоматична корекция (автономен ремонт) на структурата още при зареждане, елиминирайки дублираните или липсващите уникални идентификатори.

- **Интелигентен навигационен XPath модул:** Реализиран е мощен подмодул за заявки, който успешно съчетава филтриране по индекси и стойности на атрибути с напреднала навигация по XPath осите (`child`, `parent`, `ancestor`, `descendant`), поддържайки пълна съвместимост с XML пространствата от имена (*Namespaces*).

6.2. Архитектурни изводи и принос на шаблоните за дизайн

Разработката на софтуерната архитектура се базира изцяло на чистия обектно-ориентиран подход и модерните принципи за софтуерен дизайн (*SOLID*). Разделянето на кода на логически пакети (`models`, `io`, `parsers`, `xpath`, `commands`) доведе до ниска свързаност и висока вътрешна съвместимост на отделните модули, което улесни процеса на QA тестване и регресионна проверка.

Всички поставени изисквания по заданието бяха изпълнени успешно. Разработеният XML Parser може надеждно да чете, променя и записва XML файлове. Допълнителните функционалности като поддръжка на XPath оси и XML Namespaces правят парсера много по-мощен и доближават неговото поведение до това на професионалните библиотеки.

Внедряването на четирите класически шаблона за дизайн донесе съществени архитектурни предимства за проекта:

1. Шаблонът *Command* (Команда): Декуплира интерфейса с команден ред от бизнес логиката, превръщайки всяка потребителска операция в самостоятелен обект, което прави добавянето на нови команди изключително лесно и безопасно за съществуващия код.

2. Шаблонът *Strategy* (Стратегия): Осигури капсулация на различните алгоритми за парсане и XPath филтриране, позволявайки на системата да превключва динамично своето поведение в реално време в зависимост от обработваните данни.

3. Шаблонът *Chain of Responsibility* (Верига от отговорности): Разпредели по елегантен и йерархичен начин отговорността за разпознаване на синтактичните стъпки в XPath заявките, избягвайки сложните и трудни за поддръжка вложени условни конструкции.

4. Шаблонът *Registry* (Регистър): Чрез използването на Хеш-таблица за съхранение на уникалните идентификатори, времевата сложност за търсене на елементи бе редуцирана от линейна $O(n)$ до константно време $O(1)$, което гарантира моментално бързодействие при масивни XML бази данни.

6.3. Насоки за бъдещо разширение на софтуерната система

Въпреки че разработеното приложение е напълно завършено, стабилно и готово за експлоатация, неговата гъвкава архитектура поставя солидна основа, върху която могат да бъдат реализирани следните бъдещи разширения:

- **Графичен потребителски интерфейс (GUI):** Замяна на интерактивния интерфейс с команден ред (CLI) с модерен и интуитивен прозоречен интерфейс, изграден чрез технологията **JavaFX**. Това би позволило дървовидната структура на XML документа да бъде визуализирана графично под формата на интерактивно интелектуално дърво (*TreeView*).

- **Трансформация и експорт на данни:** Добавяне на функционалност за автоматично преобразуване на XML дървото и неговия експорт към други популярни съвременни формати за структуриран обмен на информация, като JSON (*JavaScript Object Notation*) или структурирани таблични CSV файлове.

Използвана литература

- 1) Официална документация на езика Java (Java Platform, Standard Edition 17 API Specification). Oracle Corporation.
- 2) Extensible Markup Language (XML) 1.0 (Fifth Edition) - W3C Recommendation. World Web Consortium (W3C).
- 3) <https://www.w3.org/TR/xpath/>
- 4) https://www.w3schools.com/Xml/xpath_axes.asp
- 5) https://www.w3schools.com/xml/xpath_syntax.asp
- 6) https://www.w3schools.com/xml/xml_namespaces.asp
- 7) https://www.w3schools.com/xml/xml_what_is.asp
- 8) <https://www.geeksforgeeks.org/system-design/registry-pattern/>
- 9) Лекционен курс по "Обектно-ориентирано програмиране - част 1", ТУ-Варна.

<https://github.com/fanybeytiewa/xml-parser>